

The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

Part I — Foundations

AYuSh

Contents

Part I — Foundations	6
Chapter 1: Mathematics for ML	7
Chapter 2: Python for ML Interviews.....	30
Chapter 3: Data Structures & Algorithms (ML-flavored)	56

Preface

If you are preparing for a machine learning interview in the current market, you are facing a strange paradox. The field has never been more accessible — free courses, open models, and endless tutorials — and yet interviews have never been harder to prepare for. The bar has moved. A decade ago you could pass an ML interview by explaining the bias-variance tradeoff and implementing k-means. Today you may be asked to derive backpropagation on a whiteboard at 10 a.m., design a retrieval-augmented generation system at 1 p.m., and debug a training loss curve at 4 p.m. — all for the same job.

This book exists to carry you across that entire range. It is written for three kinds of readers at once. The **complete beginner** who has decided to enter the field and wants a single, sequential path from "what is a matrix?" to "how does speculative decoding work?". The **practitioner** who uses scikit-learn or PyTorch daily but has never had to justify *why* Adam differs from RMSprop out loud, under pressure. And the **interview-ready candidate** who needs dense, honest model answers rather than hand-waving.

Three principles govern every chapter:

Nothing is assumed. Every topic begins in plain language, as if you have never met it before, and then climbs — intuition first, then formal definitions, then derivations with every symbol defined, then a small worked numeric example you can check with a pencil.

All code runs. Every code listing in this book was executed before it was printed. Where output is shown, it is real output. You can type any listing into a Python file and run it.

Interviews are the destination. Every chapter ends with real interview-style questions and model answers written at the depth an interviewer actually expects — including notes on what the interviewer is listening for.

How to use this book

The book has eight parts and is designed to be read two different ways.

The scratch-to-pro path (8-16 weeks). Read sequentially, Part I through Part VIII. Part I builds the mathematical and programming foundation; nothing later will assume more than it teaches. Do the worked examples by hand. Type out and run the code. This path turns a motivated beginner into a credible candidate.

The interview-in-two-weeks path. If you already work in the field, go straight to where roughly 80% of interview questions concentrate: Part II (classical ML), Part III (deep learning), and Part IV (NLP and LLMs), plus Chapter 34 (implement-from-scratch — the single most requested interview practice topic). Skim the Q&A sections of every other chapter, and read Part VIII the weekend before your loop.

Whichever path you take, treat the end-of-chapter Q&A as active practice, not reading material: cover the answer, say your own answer out loud, then compare. Speaking an answer is a different skill from recognizing one.

A note on the appendices: Appendix A (formula cheat sheets) and Appendix C (100 rapid-fire one-liners) are designed for the final 48 hours before an interview. Appendix E (SQL and Python quick reference) covers the data-manipulation screens that increasingly precede ML rounds.

Notation

The same symbols mean the same things in every chapter of this book. When a chapter introduces a symbol not listed here, it defines it at first use.

Symbol	Meaning
x	a single input (feature vector), $x \in \mathbb{R}^d$
X	the design matrix of all inputs, shape $n \times d$ (rows are examples)
y	the true label or target
$\hat{y}$	the model's prediction
n	number of examples; d — number of features
w, b	weight vector and bias of a single model or layer
θ	the set of all learnable parameters of a model
η	learning rate
$\mathcal{L}$	loss function (to be minimized)
$\nabla_{\theta} \mathcal{L}$	gradient of the loss with respect to parameters
$\sigma(\cdot)$	the sigmoid function $\sigma(z) = 1/(1 + e^{-z})$
$\mathbb{E}[\cdot]$	expectation; $\text{Var}(\cdot)$ — variance
$P(A B)$	conditional probability of A given B
$\mathcal{N}(\mu, \sigma^2)$	Gaussian distribution with mean μ and variance σ^2
$D_{KL}(P \ Q)$	Kullback–Leibler divergence from Q to P
$H(P)$	entropy of distribution P
$\ v\ $	Euclidean (L2) norm of vector v
A^T	transpose of matrix A ; A^{-1} — inverse
λ	regularization strength (or an eigenvalue — context makes it clear)

Vectors are column vectors unless stated otherwise. Code follows Python conventions: 0-indexed, row-major NumPy arrays where $\mathbf{X}[i]$ is the i -th example.

Part I — Foundations

Chapter 1: Mathematics for ML

Every machine learning model you will ever discuss in an interview is, underneath the API, a mathematical object: a function with parameters, fit by optimization, judged by probability. Interviewers probe math not because they enjoy calculus, but because it is the fastest way to tell whether you *understand* models or merely *use* them. A candidate who can explain why eigenvalues matter for PCA, derive the gradient of logistic loss, or say precisely what a p-value is (and is not) signals depth that no amount of framework fluency can fake.

This chapter covers the four pillars — linear algebra, calculus, probability, and statistics — plus the three topics interviewers use as depth probes: hypothesis testing, MLE vs MAP, and information theory. If you are new, read it in order; everything later in the book stands on this chapter. If you are experienced, skim the headers and go straight to the Q&A at the end — if you can answer all of it out loud, move on.

Linear algebra

Vectors: the atoms of ML

A **vector** is an ordered list of numbers. That's it. The vector $x = [5.1, 3.5, 1.4, 0.2]$ might describe a flower by four measurements. In ML, every input — an image, a sentence embedding, a customer profile — becomes a vector, because vectors are things we can do arithmetic on.

Geometrically, a vector in $\mathbb{R}^d$ is a point (or an arrow from the origin) in d -dimensional space. Two operations carry almost all of ML:

The **dot product** $x \cdot w = \sum_{i=1}^d x_i w_i$ measures alignment. If the result is large and positive the vectors point the same way; zero means perpendicular (orthogonal); negative means opposing. A linear model's prediction $\hat{y} = w^T x + b$ is nothing more than "how aligned is this input with the direction w that the model learned to associate with the positive class?"

The **norm** $\|x\| = \sqrt{\sum_i x_i^2}$ measures length. Combining the two gives **cosine similarity**:

$$\cos(\theta) = \frac{x \cdot y}{\|x\| \|y\|}$$

which strips away magnitude and keeps only direction — the standard similarity measure for text embeddings, where document length shouldn't matter but topic should.

Worked example. Let $x = [1, 2]$ and $y = [2, 1]$. Then $x \cdot y = 1 \cdot 2 + 2 \cdot 1 = 4$, $\|x\| = \|y\| = \sqrt{5}$, so $\cos(\theta) = 4/5 = 0.8$. The vectors are similar but not identical in direction — check: they are mirror images across the diagonal.

Matrices: functions, not tables

A **matrix** A of shape $m \times n$ is best understood not as a grid of numbers but as a *linear function* that maps vectors from $\mathbb{R}^n$ to $\mathbb{R}^m$ via $v \mapsto Av$. Every linear layer in a neural network is exactly this: `output = W @ input + b`. Matrix multiplication is function composition — $B(Av) = (BA)v$ — which is why a stack of linear layers *without nonlinearities* collapses into one linear layer: the product W_2W_1 is just another matrix. This single fact is the reason activation functions exist, and it is a classic interview question.

Key facts you should be able to state cold: matrix multiplication requires inner dimensions to match ($m \times n$ times $n \times p$ gives $m \times p$); it is associative but **not commutative** ($AB \neq BA$ in general); the transpose reverses order, $(AB)^T = B^T A^T$; and the inverse A^{-1} exists only for square matrices with nonzero determinant (equivalently: full rank, no zero eigenvalues, columns linearly independent — interviewers love asking for all the equivalent conditions).

Rank is the number of linearly independent columns (equivalently rows) — the true dimensionality of the space the matrix can reach. A rank-deficient design matrix X means some feature is a linear combination of others (perfect multicollinearity), and $X^T X$ becomes non-invertible — which is precisely why the normal equation for linear regression can fail and why Ridge regression adds λI to fix it. Chapter 5 builds on this.

Eigenvalues and eigenvectors: the directions a matrix cannot rotate

For a square matrix A , an **eigenvector** v is a special direction that A does not rotate — it only stretches or shrinks it:

$$Av = \lambda v$$

The stretch factor λ is the **eigenvalue**. Think of a matrix as a transformation of space: most directions get rotated somewhere else, but eigenvectors are the axes of the transformation, and eigenvalues tell you how much each axis is scaled.

Worked example. Let A be the diagonal matrix with entries 2 and 3 (rows (2, 0) and (0, 3)). Any vector along the x-axis satisfies $Av = 2v$, any vector along the y-axis satisfies $Av = 3v$. Eigenvalues are 2 and 3, eigenvectors are $[1, 0]$ and $[0, 1]$. For non-diagonal matrices the eigenvectors are not axis-aligned, but the idea is identical. To find eigenvalues by hand, solve the characteristic equation $\det(A - \lambda I) = 0$: for the matrix with rows (2, 1) and (1, 2), this is $(2 - \lambda)^2 - 1 = 0$, so $\lambda = 1$ or $\lambda = 3$ (with eigenvectors along $[1, -1]$ and $[1, 1]$ — check by multiplying).

Why ML interviews care:

PCA (Chapter 8) finds the eigenvectors of the data's covariance matrix. The eigenvector with the largest eigenvalue is the direction of maximum variance — the "principal component." Eigenvalues tell you how much variance each component explains, which is how you decide how many components to keep.

Optimization geometry. The eigenvalues of the Hessian (see calculus section) describe the curvature of the loss surface. A large ratio between the largest and smallest eigenvalue (the **condition number**) means a long, narrow valley — exactly the situation where gradient descent zig-zags and converges slowly, and the motivation for momentum and adaptive optimizers (Chapter 13).

Spectral properties of networks. Repeated multiplication by a weight matrix amplifies components along eigenvectors with $|\lambda| > 1$ and kills those with $|\lambda| < 1$ — the linear-algebra heart of exploding and vanishing gradients in RNNs (Chapter 15).

SVD: the decomposition that always exists

Eigendecomposition requires a square (and, to be well-behaved, symmetric) matrix. The **Singular Value Decomposition** works for *any* matrix, even rectangular:

$$A = U\Sigma V^T$$

where U ($m \times m$) and V ($n \times n$) are orthogonal matrices (their columns are orthonormal directions — pure rotations/reflections), and Σ is diagonal with non-negative **singular values** $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$. Read it as: *any* linear map is a rotation, followed by axis-aligned scaling, followed by another rotation. The singular values are the scaling factors, and they equal the square roots of the eigenvalues of $A^T A$.

The interview-critical application is **low-rank approximation**: keep only the top k singular values and the corresponding columns of U and V :

$$A \approx U_k \Sigma_k V_k^T$$

The Eckart-Young theorem says this is the *best* rank- k approximation of A in the least-squares sense. This is the math behind classic recommender-system matrix factorization (Chapter 30), latent semantic analysis in NLP, model-weight compression, and — because LoRA (Chapter 21) parameterizes weight *updates* as a low-rank product BA — the fine-tuning method you will most likely be asked about in an LLM interview. PCA itself is SVD applied to mean-centered data.

Other decompositions worth naming in an interview: **Cholesky** ($A = LL^T$ for symmetric positive-definite matrices; fast solving of linear systems, sampling from multivariate Gaussians) and **QR** (numerically stable least squares). You rarely need their mechanics — you need to know *when* they apply.

Calculus

Derivatives and gradients: which way is down

The derivative $f'(x)$ is the local slope: how much f changes per unit change in x . Machine learning needs exactly one big idea from calculus: **to make a loss smaller, move parameters in the direction opposite the slope**. Everything else is bookkeeping for functions of many variables.

When a function takes many inputs, $f(w_1, \dots, w_d)$, the **partial derivative** $\partial f / \partial w_i$ is the slope with respect to one input, holding the others fixed. Collecting all partials into a vector gives the **gradient**:

$$\nabla_w f = \left[\frac{\partial f}{\partial w_1}, \dots, \frac{\partial f}{\partial w_d} \right]^T$$

The gradient points in the direction of *steepest increase*, and its norm says how steep that ascent is. Gradient descent therefore updates $w \leftarrow w - \eta \nabla_w \mathcal{L}$, stepping downhill with step size η (Chapter 13 covers what happens when η is wrong).

Worked example. $f(w_1, w_2) = w_1^2 + 3w_1w_2$. Then $\partial f / \partial w_1 = 2w_1 + 3w_2$ and $\partial f / \partial w_2 = 3w_1$. At the point $(1, 2)$: $\nabla f = [8, 3]^T$. To decrease f from that point, step along $[-8, -3]$.

The chain rule: the engine of backpropagation

Deep networks are compositions of functions: $\mathcal{L} = f_3(f_2(f_1(x; w_1); w_2); w_3)$. The **chain rule** says the derivative of a composition is the product of the derivatives of the pieces:

$$\frac{d}{dx} f(g(x)) = f'(g(x)) \cdot g'(x)$$

Backpropagation (Chapter 12 derives it fully) is *just* the chain rule applied systematically, layer by layer, reusing intermediate results so nothing is computed twice. Two facts to internalize now: the chain rule multiplies local derivatives, so a chain of numbers slightly below 1 shrinks exponentially (vanishing gradients) and a chain slightly above 1 grows exponentially (exploding gradients); and computing the chain *backwards* (from loss to inputs) is cheap when the output is a scalar — one backward sweep gets every parameter's gradient at once.

Worked example. $\mathcal{L}(w) = (\sigma(wx) - y)^2$ with $x = 2$, $y = 1$, $w = 0$. Then $\sigma(0) = 0.5$. Chain: $\frac{d\mathcal{L}}{dw} = 2(\sigma(wx) - y) \cdot \sigma'(wx) \cdot x$. Since $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = 0.25$ at $z = 0$: $\frac{d\mathcal{L}}{dw} = 2(0.5 - 1)(0.25)(2) = -0.5$. The gradient is negative, so increasing w decreases the loss — which makes sense: we need $\sigma(2w)$ to move toward 1.

Jacobians and Hessians

When a function maps vectors to vectors, $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$, its derivative is the **Jacobian** — the $m \times n$ matrix of all partial derivatives $J_{ij} = \partial f_i / \partial x_j$. Each layer of a network has a Jacobian;

backprop is a product of Jacobians (multiplied against a vector, which is why it's efficient — you never materialize the full matrices).

The **Hessian** is the matrix of second derivatives of a scalar function: $H_{ij} = \partial^2 f / \partial x_i \partial x_j$. It describes *curvature*. At a critical point (zero gradient): all Hessian eigenvalues positive → local minimum; all negative → local maximum; mixed signs → saddle point. In high dimensions saddle points vastly outnumber local minima, which is part of why plain gradient descent works better in deep learning than early theory predicted — most zero-gradient traps are saddles you can escape, not minima you're stuck in.

The Hessian's eigenvalue spread (condition number, again) controls how hard the optimization is; Newton's method uses $H^{-1}\nabla f$ to rescale steps by curvature but is $O(d^2)$ memory and $O(d^3)$ compute per step — hopeless for a model with millions of parameters, which is why deep learning uses first-order optimizers with cheap curvature *approximations* like Adam's per-parameter second-moment estimates (Chapter 13).

Probability

The rules of uncertainty

Probability is the mathematics of consistent reasoning under uncertainty. Three rules generate everything else. **Non-negativity and normalization**: probabilities are in $[0, 1]$ and sum to 1 over all outcomes. The **sum rule** (marginalization): $P(A) = \sum_B P(A, B)$ — if you don't care about B , sum it out. The **product rule**: $P(A, B) = P(A | B)P(B)$ — a joint probability factors into a conditional times a marginal.

Conditional probability $P(A | B) = P(A, B)/P(B)$ is the probability of A *within the world where B happened*. Conditioning is re-normalizing to a smaller universe. Two events are **independent** when conditioning tells you nothing: $P(A | B) = P(A)$, equivalently $P(A, B) = P(A)P(B)$.

Bayes' theorem: inverting the question

Rearranging the product rule gives the most important equation in applied statistics:

$$P(H | D) = \frac{P(D | H)P(H)}{P(D)}$$

Read: the **posterior** probability of hypothesis H given data D equals the **likelihood** of the data under the hypothesis, times the **prior** probability of the hypothesis, normalized by the **evidence** $P(D)$. Bayes' theorem converts "how likely is this data if the hypothesis is true?" (easy to model) into "how likely is the hypothesis given this data?" (what you actually want).

Worked example — the one interviewers actually ask. A disease affects 1% of the population. A test has 95% sensitivity ($P(+ | \text{sick}) = 0.95$) and 90% specificity ($P(- | \text{healthy}) = 0.90$). You test positive. What is the probability you are sick?

$$P(\text{sick} | +) = \frac{0.95 \times 0.01}{0.95 \times 0.01 + 0.10 \times 0.99} = \frac{0.0095}{0.0095 + 0.099} \approx 0.088$$

About 9%, not 95%. The prior (1% base rate) dominates because false positives from the huge healthy population swamp true positives from the tiny sick population. The lesson — **never ignore the base rate** — is the core intuition behind why accuracy is a useless metric on imbalanced data (Chapter 10) and why Naive Bayes works the way it does (Chapter 6).

Distributions you must know

A **random variable** is a quantity whose value is uncertain; its **distribution** assigns probabilities to its possible values (a probability *mass* function for discrete variables, a *density* for continuous ones).

Bernoulli(p) — a single yes/no trial: $P(X = 1) = p$, mean p , variance $p(1 - p)$. The output of a binary classifier is a Bernoulli parameter; log loss is exactly the Bernoulli negative log-likelihood.

Binomial(n, p) — the number of successes in n independent Bernoulli trials: $P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$, mean np , variance $np(1 - p)$. This is the distribution of "how many of my 10,000 users clicked," which makes it the backbone of A/B test analysis.

Poisson(λ) — counts of rare events in a fixed window when events are independent: $P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$, with mean *and* variance both λ (a fact interviewers check). It is the limit of Binomial(n, p) as $n \rightarrow \infty$, $p \rightarrow 0$ with $np = \lambda$ fixed. Used for arrival counts: requests per second, fraud events per day.

Gaussian $\mathcal{N}(\mu, \sigma^2)$ — the bell curve:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

Why it is everywhere: (1) the Central Limit Theorem (below) makes sums of independent effects approximately Gaussian; (2) it is the maximum-entropy distribution for a fixed mean and variance — the "least assuming" choice; (3) assuming Gaussian noise in a regression model makes maximum likelihood *identical* to minimizing squared error, a derivation done in the MLE section below and asked verbatim in interviews. Remember the 68-95-99.7 rule: those are the probabilities of landing within 1, 2, and 3 standard deviations of the mean.

Worked examples with the distributions.

Binomial. A model serves 1,000 users; each independently converts with probability 0.1. Expected conversions: $np = 100$. Standard deviation: $\sqrt{np(1 - p)} = \sqrt{90} \approx 9.5$. So a day

with 80 conversions is about two standard deviations low — unusual but not impossible ($\approx 2.5\%$ of days by the normal approximation); a day with 50 is a six-sigma event, and you should suspect the pipeline, not chance. This "expected $\pm$ a few SDs" reflex is the fastest sanity check in applied ML.

Poisson. A fraud system sees on average $\lambda = 3$ fraudulent transactions per hour. Probability of a completely clean hour: $P(X = 0) = e^{-3} \approx 0.05$. Probability of 8 or more (an alarm threshold): small enough ($\sim 1\%$) that alerting on it has a tolerable false-alarm rate. Because Poisson variance equals its mean, a monitoring stream whose counts vary much more than their mean (**overdispersion**) violates the Poisson assumption — usually a sign of clustered or bursty events, and a favorite trick question.

Gaussian and z-scores. Model latency is roughly $\mathcal{N}(120, 15^2)$ milliseconds. What fraction of requests exceed 150 ms? Standardize: $z = (150 - 120)/15 = 2$, and $P(Z > 2) \approx 2.3\%$. Standardizing to z-scores — subtract the mean, divide by the SD — is the same operation as feature standardization in Chapter 9; it puts any Gaussian on the common $\mathcal{N}(0, 1)$ scale where tables and intuition apply.

Statistics

Describing data

The **mean** $\bar{x} = \frac{1}{n} \sum_i x_i$ is the balance point; it minimizes squared distance to the data but is dragged by outliers. The **median** (middle value) minimizes absolute distance and resists outliers — for skewed data like incomes or latencies, median (and percentiles like p95/p99) is the honest summary. The **mode** is the most frequent value. In a right-skewed distribution: mode < median < mean — a favorite quick interview check.

Variance measures spread: $\text{Var}(X) = \mathbb{E}[(X - \mu)^2] = \mathbb{E}[X^2] - \mu^2$. Its square root, the **standard deviation** σ , is in the same units as the data. When estimating from a sample, divide by $n - 1$ instead of n (**Bessel's correction**): the sample mean is itself fit to the data and "uses up" one degree of freedom, making raw $1/n$ variance systematically too small.

Covariance measures how two variables move together: $\text{Cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)]$. Its scale-free version is the **Pearson correlation** $\rho = \text{Cov}(X, Y)/(\sigma_X \sigma_Y) \in [-1, 1]$. Three traps to name in interviews: correlation measures only *linear* relationships (a perfect parabola can have $\rho = 0$); correlation is not causation (confounders, selection bias); and zero correlation does not imply independence (though independence does imply zero correlation). The covariance *matrix* of a dataset — $\Sigma = \frac{1}{n} X_c^T X_c$ for centered data — is the object PCA eigendecomposes.

Law of Large Numbers and the Central Limit Theorem

The **Law of Large Numbers** says the sample mean converges to the true mean as n grows: averages stabilize. It is why Monte Carlo methods work, why more data reduces estimation noise, and why your A/B test needs enough samples before its metric means anything.

The **Central Limit Theorem** is stronger and stranger: the *distribution* of the sample mean approaches a Gaussian, no matter what distribution the individual samples come from (as long as they are independent with finite variance):

$$\bar{X}_n \sim \mathcal{N}(\mu, \frac{\sigma^2}{n})$$

The spread of the sample mean — the **standard error** $\sigma/\sqrt{n}$ — shrinks with $\sqrt{n}$, which is the reason halving your error bars requires *four times* the data, and the mathematical foundation of every confidence interval and significance test in the next section. LLN says *where* the average goes; CLT says *how it fluctuates* on the way.

Hypothesis testing

The logic of a hypothesis test is proof by statistical contradiction. State a **null hypothesis** H_0 ("the new model is no better than the old one"). Assume it is true. Compute how surprising the observed data would be under that assumption. If the data is too surprising, reject H_0 .

The **p-value** is the probability, *assuming H_0 is true*, of observing data at least as extreme as what you saw. It is **not** the probability that H_0 is true — that misstatement is the single most common statistics error in interviews, and interviewers bait it deliberately. A p-value below a pre-chosen threshold α (conventionally 0.05) means "reject H_0 "; α is exactly the **Type I error** rate — the probability of rejecting a true null (false alarm). **Type II error** (β) is failing to reject a false null (missed detection), and **power** $= 1 - \beta$ is the probability of catching a real effect. Power grows with effect size and sample size; that is why you run a *power analysis before* an experiment to decide how much data you need.

The t-test compares means. The one-sample statistic is $t = (\bar{x} - \mu_0)/(s/\sqrt{n})$: how many standard errors is the sample mean from the hypothesized mean? The two-sample version compares two groups (e.g., treatment vs control conversion values). It assumes approximately normal sample means (CLT gives you this for large n) and, in the classic form, similar variances (Welch's variant relaxes this — say "Welch's t-test" in interviews when unsure).

The chi-square test handles categorical counts: $\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$ over cells of a contingency table, where O is observed and E is the count expected under independence. Use it for "did signup rate differ across these five landing pages?" or goodness-of-fit checks.

Confidence intervals. A 95% CI for a mean is $\bar{x} \pm 1.96 \cdot s/\sqrt{n}$. Precise meaning (another bait question): if you repeated the experiment many times, 95% of the intervals constructed this way would contain the true value. It is a statement about *the procedure*, not "there's a 95% chance the truth is in this particular interval."

Power analysis, worked. How many users per arm to detect a lift from 10.0% to 10.8% conversion at $\alpha = 0.05$ (two-sided) with 80% power? The standard formula for two proportions is

$$n \approx \frac{(z_{1-\alpha/2} + z_{1-\beta})^2 \cdot 2\bar{p}(1-\bar{p})}{(p_B - p_A)^2}$$

with $z_{0.975} = 1.96$, $z_{0.80} = 0.84$, $\bar{p} = 0.104$: $n \approx \frac{(2.8)^2 \times 2 \times 0.0932}{(0.008)^2} \approx 22,800$ per arm. This is why the 10,000-per-arm experiment in the code section below *correctly* fails to find the effect — it was never big enough to see it. Memorize the structure, not the constants: required n scales with variance and *inversely with the square* of the effect size — detecting an effect half as big costs four times the users.

A/B testing packages all of this: randomize users into control (A) and treatment (B), pre-register the metric and α , run the power analysis, collect data, run the test (t-test for continuous metrics, chi-square or two-proportion z-test for conversion rates). Practical failure modes interviewers probe: **peeking** (checking repeatedly and stopping when significant inflates false positives far above α); **multiple comparisons** (testing 20 metrics at $\alpha = 0.05$ expects one false positive — use Bonferroni or FDR corrections); **novelty effects** (users click new things briefly); and **interference** (one user's treatment affecting another's behavior, e.g., in marketplaces or social feeds). Chapter 27 revisits A/B testing for model deployment.

Maximum likelihood and MAP

MLE: let the data speak

You have data and a model with unknown parameters θ . **Maximum Likelihood Estimation** picks the θ that makes the observed data most probable:

$$\hat{\theta}_{MLE} = \arg \max_{\theta} P(D|\theta) = \arg \max_{\theta} \prod_{i=1}^n P(x_i | \theta)$$

The product (from assuming independent samples) turns into a sum under a logarithm — which is monotonic, so maximizing log-likelihood is equivalent and numerically far saner:

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \sum_{i=1}^n \log P(x_i | \theta)$$

Worked example (coin). Flip a coin 10 times, see 7 heads. Likelihood:

$L(p) = p^7(1-p)^3$. Log-likelihood: $\ell(p) = 7\log p + 3\log(1-p)$. Set the derivative to zero:

$7/p - 3/(1-p) = 0 \Rightarrow p = 0.7$. MLE just returns the empirical frequency — data speaks, nothing else does.

The derivation interviewers actually want. Assume regression targets are generated as $y_i = w^T x_i + \varepsilon_i$ with Gaussian noise $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$. The log-likelihood of the data is

$$\ell(w) = \sum_i \log \mathcal{N}(y_i | w^T x_i, \sigma^2) = -\frac{1}{2\sigma^2} \sum_i (y_i - w^T x_i)^2 + \text{const}$$

Maximizing it is *exactly* minimizing the sum of squared errors. **MSE is not an arbitrary choice — it is Gaussian-noise MLE.** Likewise, cross-entropy loss is Bernoulli/categorical MLE. Nearly every loss function you will ever use is a negative log-likelihood in disguise; saying this unprompted is a strong interview signal.

MAP: data plus belief

MLE can overfit small samples (see 3 heads in 3 flips $\rightarrow \hat{p} = 1$, a certainty no reasonable person holds). **Maximum A Posteriori** estimation multiplies in a prior over parameters and maximizes the posterior:

$$\hat{\theta}_{MAP} = \arg \max_{\theta} P(D|\theta)P(\theta) = \arg \max_{\theta} [\log P(D|\theta) + \log P(\theta)]$$

The prior acts as a penalty on implausible parameters. The connection interviewers prize: **a Gaussian prior on weights gives L2 regularization (Ridge); a Laplace prior gives L1 (Lasso).** Take $P(w) = \mathcal{N}(0, \tau^2)$: $\log P(w) = -\|w\|^2/(2\tau^2) + \text{const}$, so MAP = MLE loss + $\lambda\|w\|^2$ with $\lambda = \sigma^2/\tau^2$. Regularization is not a hack — it is a prior belief that weights should be small. As $n \rightarrow \infty$ the likelihood term dominates and MAP converges to MLE: with enough data, the prior stops mattering.

Information theory

Entropy: the price of uncertainty

The **surprise** of an outcome with probability p is $-\log p$ — rare events are more surprising, certain events not at all. **Entropy** is expected surprise:

$$H(P) = -\sum_x P(x) \log P(x)$$

With log base 2, entropy is measured in bits and has an operational meaning: the minimum average number of bits needed to encode samples from P . A fair coin has $H = 1$ bit; a coin with $p = 0.99$ has $H \approx 0.08$ bits — nearly no information because outcomes are nearly certain. Uniform distributions maximize entropy; deterministic ones have zero. Decision trees (Chapter 6) split on features that maximally *reduce* entropy of the label — information gain.

Cross-entropy and KL divergence

Cross-entropy is the expected surprise when reality follows P but you encode using model Q :

$$H(P, Q) = -\sum_x P(x) \log Q(x)$$

Training a classifier minimizes exactly this, with P the one-hot true label and Q the softmax output — which collapses to $-\log Q(\text{true class})$: penalize the model by how surprised it is at the truth.

KL divergence is the *excess* cost of using the wrong distribution:

$$D_{KL}(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} = H(P, Q) - H(P)$$

Properties to state in interviews: $D_{KL} \geq 0$, zero iff $P = Q$; and it is **asymmetric** — $D_{KL}(P\|Q) \neq D_{KL}(Q\|P)$, so it is not a distance metric. Since $H(P)$ is fixed by the data, minimizing cross-entropy is minimizing KL divergence from the model to the truth. KL appears throughout the modern stack: the regularizer in VAEs (Chapter 18), the policy constraint in PPO and RLHF (Chapters 21, 32), and knowledge distillation losses.

Mutual information measures how much knowing one variable reduces uncertainty about another: $I(X; Y) = H(X) - H(X | Y) = D_{KL}(P(X, Y) \| P(X)P(Y))$. It is zero iff X and Y are independent, and unlike correlation it captures *nonlinear* dependence — the principled version of "are these two features related?", used in feature selection (Chapter 9).

Code implementations

Every listing below was executed as printed; outputs are real. Interviewers frequently ask for exactly these implementations "from scratch, no sklearn."

PCA via eigendecomposition — and via SVD

```
import numpy as np
rng = np.random.default_rng(42)

X = rng.normal(size=(200, 2)) @ np.array([[3.0, 0.0], [1.0, 0.5]]) # correlated data
Xc = X - X.mean(axis=0) # 1. center the data
cov = (Xc.T @ Xc) / (len(Xc) - 1) # 2. covariance matrix (d x d)
eigvals, eigvecs = np.linalg.eigh(cov) # 3. eigendecomposition (symmetric -> eigh)
order = np.argsort(eigvals)[::-1] # 4. sort by descending eigenvalue
eigvals, eigvecs = eigvals[order], eigvecs[:, order]
explained = eigvals / eigvals.sum()
Z = Xc @ eigvecs[:, :1] # 5. project onto top-1 component
print("eigenvalues:", np.round(eigvals, 3))
print("explained variance ratio:", np.round(explained, 3))
U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
print("PC1 from eigh:", np.round(eigvecs[:, 0], 3), "| PC1 from SVD:", np.round(Vt[0], 3))
```

Output:

```
eigenvalues: [9.992 0.184]
explained variance ratio: [0.982 0.018]
PC1 from eigh: [-0.999 -0.051] | PC1 from SVD: [0.999 0.051]
```

Line by line: centering matters because PCA finds directions of variance *around the mean* — skip it and the first component points at the mean instead of along the data. We use `eigh` (not `eig`) because the covariance matrix is symmetric, which guarantees real eigenvalues and lets NumPy use a faster, stabler algorithm. The projection `Xc @ eigvecs[:, :1]` compresses 2-D data to 1-D while keeping 98.2% of the variance. The SVD route gives the same component — with an arbitrary sign flip, which is expected: if v is an eigenvector so is $-v$, and mentioning that sign ambiguity in an interview shows real fluency.

Gradient descent = Gaussian MLE, verified against the normal equation

```
import numpy as np
rng = np.random.default_rng(0)
n, d = 500, 3
X = rng.normal(size=(n, d))
true_w = np.array([2.0, -1.0, 0.5])
y = X @ true_w + rng.normal(scale=0.3, size=n)

w, eta = np.zeros(d), 0.1
for step in range(200):
    grad = -2 / n * X.T @ (y - X @ w)  # d(MSE)/dw, derived via the chain rule
    w -= eta * grad
w_closed = np.linalg.solve(X.T @ X, X.T @ y)  # normal equation
print("gradient descent w:", np.round(w, 3))
print("normal equation w :", np.round(w_closed, 3))
```

Output:

```
gradient descent w: [ 2.015 -0.998  0.519]
normal equation w : [ 2.015 -0.998  0.519]
```

The gradient line is the chain rule from this chapter applied to $\mathcal{L}(w) = \frac{1}{n} \|y - Xw\|^2$. Note `np.linalg.solve` rather than explicitly inverting $X^T X$ — inverting is slower and numerically worse, and saying so is an easy interview point. Both estimates recover `true_w` up to noise, and they agree to three decimals: the iterative and closed-form solutions of the same convex problem.

MLE for a coin, numerically

```
import numpy as np
flips = np.array([1,1,1,0,1,1,0,1,0,1])  # 7 heads out of 10
ps = np.linspace(0.01, 0.99, 99)
```

```
loglik = flips.sum()*np.log(ps) + (len(flips)-flips.sum())*np.log(1-ps)
print("MLE p-hat:", np.round(ps[np.argmax(loglik)], 2))
```

Output:

```
MLE p-hat: 0.7
```

The grid search lands exactly on the calculus answer 7/10 from the worked example — a useful sanity-check pattern: when you derive an estimator by hand, verify it numerically.

Entropy, cross-entropy, and KL divergence from scratch

```
import numpy as np
def entropy(p):
    p = np.asarray(p); return -np.sum(p * np.log2(p, where=p>0))
def cross_entropy(p, q):
    p, q = np.asarray(p), np.asarray(q); return -np.sum(p*np.log2(q, where=q>0))
def kl(p, q):
    return cross_entropy(p, q) - entropy(p)

P = [0.5, 0.5]; Q = [0.9, 0.1]
print("H(P) =", entropy(P), "bits | H(P,Q) =", round(cross_entropy(P,Q),3),
      "| KL(P||Q) =", round(kl(P,Q),3))
print("KL(Q||P) =", round(kl(Q,P),3), " (asymmetric!)")
```

Output:

```
H(P) = 1.0 bits | H(P,Q) = 1.737 | KL(P||Q) = 0.737
KL(Q||P) = 0.531 (asymmetric!)
```

The `where=p>0` guard implements the convention $0 \log 0 = 0$ without NaNs. The two KL values differ — the asymmetry stated in the theory section, demonstrated in four lines.

An A/B test from scratch — including its failure

```
import numpy as np
from math import erf, sqrt
rng = np.random.default_rng(7)
nA, nB = 10_000, 10_000
conv_A = rng.binomial(1, 0.100, nA) # control: 10.0% true rate
conv_B = rng.binomial(1, 0.108, nB) # treatment: 10.8% true rate
pA, pB = conv_A.mean(), conv_B.mean()
p_pool = (conv_A.sum() + conv_B.sum()) / (nA + nB)
se = np.sqrt(p_pool * (1 - p_pool) * (1/nA + 1/nB))
z = (pB - pA) / se
p_value = 2 * (1 - 0.5 * (1 + erf(abs(z) / sqrt(2)))) # two-sided, normal CDF
ci = (pB - pA - 1.96*se, pB - pA + 1.96*se)
print(f"control {pA:.4f}, treatment {pB:.4f}, lift {pB-pA:+.4f}")
print(f"z = {z:.3f}, p-value = {p_value:.4f}")
print(f"95% CI for lift: [{ci[0]:+.4f}, {ci[1]:+.4f}]")
```

Output:

```
control 0.1042, treatment 0.1059, lift +0.0017
z = 0.392, p-value = 0.6950
95% CI for lift: [-0.0068, +0.0102]
```

Read this output carefully, because it is the most instructive failure in the chapter: the treatment *really is* better (10.8% vs 10.0% by construction), yet the test says "not significant" ($p = 0.70$). Nothing is wrong with the code — the experiment is **under-powered**. Detecting a 0.8-point lift on a 10% base rate at 80% power needs roughly 30,000+ users per arm, not 10,000. This is exactly the scenario behind the interview question "your A/B test shows no effect — what do you conclude?" The correct answer: *absence of evidence is not evidence of absence; check the power and the CI* — and indeed the confidence interval $[-0.007, +0.010]$ happily contains the true lift of $+0.008$.

The CLT, watched directly

```
import numpy as np
rng = np.random.default_rng(7)
raw = rng.exponential(scale=1.0, size=(100_000,)) # very non-Gaussian
means = rng.exponential(scale=1.0, size=(100_000, 50)).mean(axis=1)
print("raw skewness :", round(float(((raw-raw.mean())**3).mean() / raw.std()**3), 2))
print("means skewness:", round(float(((means-means.mean())**3).mean() /
means.std()**3), 2))
print("std of means :", round(float(means.std()), 4), "~ sigma/sqrt(n) =", round(1/
np.sqrt(50), 4))
```

Output:

```
raw skewness : 1.97
means skewness: 0.3
std of means : 0.1411 ~ sigma/sqrt(n) = 0.1414
```

Exponential samples are heavily skewed (1.97), but averages of just 50 of them are already nearly symmetric (0.3) — the CLT in action — and their spread matches the predicted standard error $\sigma/\sqrt{n}$ to three decimals.

Pitfalls, comparisons and practical tips

Confusion	Resolution
Eigendecomposition vs SVD	Eigen: square matrices, $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$. SVD: any matrix, $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^T$. For symmetric PSD matrices (covariance), they coincide. PCA can be computed either way; SVD on centered data is more numerically stable.
Gradient vs Jacobian vs Hessian	Gradient: scalar function, vector of first derivatives. Jacobian: vector function, matrix of first derivatives. Hessian: scalar function, matrix of <i>second</i> derivatives (curvature).

Confusion	Resolution
Likelihood vs probability	Probability: fixed parameters, varying data. Likelihood: fixed data, varying parameters. Same formula, opposite viewpoint.
p-value	It is $P(\text{data at least this extreme} \mid H_0)$ — not $P(H_0 \mid \text{data})$, not the probability the result is a fluke, not effect size.
Confidence interval	A statement about the procedure's long-run coverage — not "95% chance the truth is in this interval."
MLE vs MAP	MAP = MLE + prior. Gaussian prior $\rightarrow$ L2 penalty; Laplace prior $\rightarrow$ L1. They converge as $n \rightarrow \infty$.
Cross-entropy vs KL	$H(P, Q) = H(P) + D_{KL}(P \parallel Q)$. Minimizing one minimizes the other when P is fixed (labels).
Correlation vs independence	Independence $\implies$ zero correlation. Zero correlation $\nRightarrow$ independence (nonlinear dependence invisible to Pearson). Mutual information catches it.
Variance $1/n$ vs $1/(n-1)$	Sample variance uses $n-1$ (Bessel) because the sample mean consumed one degree of freedom.

Practical tips: when asked to derive anything, start by *defining every symbol* — interviewers grade setup as much as algebra. When numbers are involved (Bayes, tests), write the base rates first. And when you finish a derivation, sanity-check limiting cases out loud ($n \rightarrow \infty$, $p \rightarrow 0$, $\lambda \rightarrow 0$) — it catches sign errors and reads as maturity.

Interview questions and answers

Conceptual

Q1. What are eigenvalues and eigenvectors, intuitively, and where do they show up in ML?

An eigenvector of a matrix is a direction the matrix doesn't rotate — it only scales it, and the eigenvalue is that scale factor. They are the natural axes of a linear transformation. In ML: PCA takes the eigenvectors of the covariance matrix as directions of maximal variance; Hessian eigenvalues describe loss-surface curvature and condition optimization; and powers of weight matrices amplify or kill signal along eigen-directions, explaining exploding/vanishing gradients in RNNs.

Interviewers listen for: the geometric one-liner plus at least two concrete ML applications.

Q2. Why does PCA use the covariance matrix specifically?

PCA seeks the direction $\mathbf{v}$ (unit norm) maximizing the variance of the projected data $\text{Var}(X\mathbf{v}) = \mathbf{v}^T \Sigma \mathbf{v}$, where Σ is the covariance matrix. Maximizing $\mathbf{v}^T \Sigma \mathbf{v}$ subject to $\|\mathbf{v}\| = 1$ is solved (via a Lagrange multiplier) exactly by the top eigenvector of Σ , with the eigenvalue equal to the variance captured. So the covariance matrix isn't a choice — it falls out of the objective "find high-variance directions."

Q3. What is the difference between eigendecomposition and SVD, and when would you use each?

Eigendecomposition applies to square matrices and can involve complex values unless the matrix is symmetric; SVD applies to any matrix and always exists with real non-negative singular values. For a symmetric PSD matrix like a covariance, the two coincide. Use SVD for rectangular data matrices, low-rank approximation (Eckart-Young optimality), and numerical stability; use eigendecomposition when the object of interest is inherently square and symmetric.

Q4. Why can't a stack of linear layers without activations learn anything a single layer can't?

Because matrix multiplication composes into a matrix: $W_2(W_1\mathbf{x}) = (W_2W_1)\mathbf{x}$, which is a single linear map. Depth without nonlinearity adds parameters but no expressive power. Nonlinear activations break the composition, letting depth build genuinely new function classes.

Q5. What does the condition number of a matrix tell you, and why should someone training models care?

It's the ratio of largest to smallest singular value (for the Hessian: eigenvalue). A large condition number means the loss surface is a long narrow valley: the gradient points mostly across the valley rather than along it, so gradient descent zig-zags and you must use a learning rate small enough for the steepest direction, making progress in the flattest direction glacial. This motivates momentum, Adam, second-order approximations, feature standardization, and batch norm.

Q6. Explain the difference between the Law of Large Numbers and the Central Limit Theorem.

LLN: the sample mean converges to the true mean — it tells you *where* averages go, but nothing about fluctuation. CLT: the distribution of the sample mean approaches a Gaussian with standard deviation $\sigma/\sqrt{n}$, regardless of the underlying distribution (finite variance, independence) — it tells you the *shape and size* of the remaining error, enabling confidence intervals and tests.

Q7. What is entropy, and what does it mean for a distribution to have high or low entropy?

Entropy is expected surprise, $H = -\sum p \log p$ — operationally, the minimum average bits to encode outcomes. Uniform distributions maximize it (nothing is predictable); deterministic ones have zero. In ML it appears as the impurity measure in decision trees, the target of label smoothing, and the exploration bonus in RL. A softmax output with high entropy means the model is unsure.

Q8. Why is KL divergence not a distance metric, and does it matter?

It's asymmetric ($D_{KL}(P\|Q) \neq D_{KL}(Q\|P)$) and violates the triangle inequality. It matters practically: minimizing $D_{KL}(P\|Q)$ over Q (forward KL, as cross-entropy training does) is mean-seeking — Q spreads to cover all of P 's mass; minimizing $D_{KL}(Q\|P)$ (reverse KL, as in variational inference) is mode-seeking — Q locks onto one mode. Same "distance," qualitatively different fitted models.

Mathematical / derivation

Q9. Derive why minimizing MSE is equivalent to maximum likelihood under Gaussian noise.

Assume $y_i = f_w(x_i) + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ i.i.d. Then

$P(y_i | x_i, w) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp(-(y_i - f_w(x_i))^2 / 2\sigma^2)$. Log-likelihood:

$\ell(w) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_i (y_i - f_w(x_i))^2$. The first term is constant in w , so

maximizing ℓ is minimizing $\sum_i (y_i - f_w(x_i))^2$ — the sum of squared errors.

Interviewers listen for: stating the noise assumption up front, and noting the constant term drops.

Q10. Show that MAP with a Gaussian prior gives L2 regularization.

MAP maximizes $\log P(D | w) + \log P(w)$. With prior $w \sim \mathcal{N}(0, \tau^2 I)$:

$\log P(w) = -\frac{\|w\|^2}{2\tau^2} + c$. So MAP minimizes $\text{NLL}(w) + \frac{1}{2\tau^2} \|w\|^2$ — squared-error (or any) loss plus an L2 penalty with $\lambda = \sigma^2 / \tau^2$ for Gaussian likelihood. Tighter prior (small τ) $\rightarrow$ bigger $\lambda \rightarrow$ stronger shrinkage. A Laplace prior $P(w) \propto e^{-|w|/b}$ gives the L1 penalty the same way.

Q11. Derive the MLE for the parameter of a Bernoulli distribution.

Given k successes in n trials, $\ell(p) = k \log p + (n - k) \log(1 - p)$. Setting

$\ell'(p) = k/p - (n - k)/(1 - p) = 0$ gives $k(1 - p) = (n - k)p$, so $\hat{p} = k/n$: the sample frequency. Check the second derivative is negative (it is, for $p \in (0, 1)$), confirming a maximum.

Q12. A test for a condition with 2% prevalence has 90% sensitivity and 95% specificity. Compute the probability that a person testing positive actually has the condition.

$P(\text{sick} | +) = \frac{0.90 \times 0.02}{0.90 \times 0.02 + 0.05 \times 0.98} = \frac{0.018}{0.018 + 0.049} = \frac{0.018}{0.067} \approx 0.27$. Despite an accurate-sounding test, only $\sim 27\%$ of positives are true — the 98% healthy majority generates most of the positives. Always compute the denominator as (true positives + false positives).

Q13. Compute the entropy of a fair die, and of a die that always rolls 6.

Fair: $H = -\sum_{i=1}^6 \frac{1}{6} \log_2 \frac{1}{6} = \log_2 6 \approx 2.585$ bits — the maximum for 6 outcomes.

Deterministic: $H = -1 \log_2 1 = 0$ bits. Entropy measures spread of probability, not number of faces.

Q14. Why does sample variance divide by $n-1$?

Because deviations are measured from the *sample* mean, which was itself chosen to minimize those squared deviations — so $\sum (x_i - \bar{x})^2$ is systematically smaller than $\sum (x_i - \mu)^2$. Taking expectations shows $\mathbb{E}[\sum (x_i - \bar{x})^2] = (n-1)\sigma^2$, so dividing by $n-1$ makes the estimator unbiased. One degree of freedom was spent estimating the mean.

Q15. What is the gradient of $f(w) = \|y - Xw\|^2$, and what do you get when you set it to zero?

Expand: $f = (y - Xw)^T (y - Xw)$. Gradient: $\nabla_w f = -2X^T(y - Xw)$. Setting to zero: $X^T X w = X^T y$ — the normal equation, giving $\hat{w} = (X^T X)^{-1} X^T y$ when $X^T X$ is invertible (full column rank). If features are collinear it isn't; Ridge adds λI making $X^T X + \lambda I$ always invertible.

Statistical reasoning**Q16. Define p-value precisely. Your A/B test gives $p = 0.03$ — what can and can't you say?**

The p-value is the probability, computed assuming the null hypothesis is true, of data at least as extreme as observed. With $p = 0.03$ you can say: "if there were truly no effect, results this extreme would occur 3% of the time; at $\alpha = 0.05$ we reject the null." You cannot say: there's a 97% chance the treatment works, or the effect is large, or the finding will replicate. Effect size comes from the estimate and CI, not the p-value.

Q17. What are Type I and Type II errors, and how do they trade off?

Type I: rejecting a true null (false positive), rate α . Type II: failing to reject a false null (missed effect), rate β ; power = $1 - \beta$. Lowering α (stricter threshold) raises β for fixed data. You escape the tradeoff with more samples or bigger effects. Which error is worse is a product question: flagging fraud (Type I annoys users) vs missing fraud (Type II loses money).

Q18. Your A/B test shows no significant difference. The PM says "so the feature has no effect." How do you respond?

Non-significance is not evidence of no effect — the test may be under-powered. Check: (1) the confidence interval — if it includes effect sizes that would matter for the business, the test is inconclusive, not negative; (2) the pre-experiment power analysis — did we have the sample size to detect the minimum effect we care about? (3) data quality — randomization balance, metric definition. The chapter's Listing 5 shows a genuinely better treatment failing to reach significance at $n=10,000/\text{arm}$.

Q19. Why is peeking at an A/B test repeatedly and stopping when it hits significance a problem?

Each look is another chance for noise to cross the threshold; the actual false-positive rate across many peeks far exceeds the nominal α (with enough looks a null result will eventually "reach" $p < 0.05$ by chance). Fixes: fix the horizon in advance, use sequential testing methods designed for continuous monitoring (alpha-spending, mSPRT), or Bayesian approaches with explicit stopping rules.

Q20. When would you use a chi-square test vs a t-test?

t-test: comparing means of a continuous metric between groups (revenue per user, session length), relying on CLT normality of means. Chi-square: association between categorical variables from count data (converted/not $\times$ variant A/B/C), comparing observed to expected counts. For a 2×2 conversion table, the chi-square test is equivalent to the two-proportion z-test squared.

Q21. Correlation between two features is 0. A colleague concludes they're unrelated. What's wrong?

Pearson correlation captures only linear relationships. $Y = X^2$ with symmetric X has exactly zero correlation but complete dependence. Check scatterplots, rank correlation (Spearman) for monotone-nonlinear cases, or mutual information for general dependence. Only independence guarantees zero correlation, not the converse.

Applied / scenario**Q22. You must reduce 10,000 features to a few hundred for a downstream model. Walk through using SVD/PCA, including the decisions involved.**

Standardize features first (PCA chases variance — unscaled features with big units dominate). Compute truncated SVD of the centered matrix (never form the $10k \times 10k$ covariance explicitly; use randomized/truncated solvers). Choose k by cumulative explained variance (e.g., 95%), the eigenvalue elbow, or — best — downstream validation performance. Caveats to volunteer: components are linear and may destroy interpretability; PCA is unsupervised, so top-variance directions aren't necessarily predictive; fit the PCA on training data only to avoid leakage.

Q23. Training loss decreases smoothly, then explodes to NaN. Which mathematical concepts from this chapter diagnose it?

Exploding gradients: the chain rule multiplies per-layer Jacobians, and if their spectral norms exceed 1 the product grows exponentially; a large learning rate then turns a big gradient into a divergent step (η too large relative to local curvature — largest Hessian eigenvalue). Numerically, e^z in softmax/sigmoid overflows for large logits. Fixes: gradient clipping, lower η or warmup, normalization layers, numerically stable log-sum-exp implementations.

Q24. Metrics from a small user segment are extremely volatile week to week. A stakeholder wants to act on the latest week's number. What do you tell them?

Small n means large standard error ($\sigma/\sqrt{n}$): the weekly figure is mostly noise around the true rate. Quantify it — show the CI around the weekly estimate; it will likely span the entire "movement" being reacted to. Recommend aggregating weeks, hierarchical/shrinkage estimates toward the global mean (the Bayesian view: small samples should borrow strength from priors), and defining a minimum sample size before the metric is reported.

Q25. Why does cross-entropy loss, rather than accuracy, get optimized during classifier training?

Accuracy is a step function of the parameters — zero gradient almost everywhere, so gradient descent can't use it. Cross-entropy is smooth, aligns with MLE of the categorical likelihood, and its gradient w.r.t. the logits (softmax — one-hot) is simple and well-scaled: confident wrong predictions get large gradients, near-correct ones small. You optimize the differentiable surrogate, then report the step-function business metric.

Q26. You fit $p = 1.0$ for "heads" after seeing 3 heads in 3 flips. What went wrong, and what's the fix?

That's the MLE doing what it does — matching the empirical frequency — with too little data. It assigns zero probability to tails, an infinitely confident claim from 3 observations (and it breaks anything downstream taking $\log(0)$). Fix with a prior: MAP with a Beta prior gives $\hat{p} = (k + \alpha - 1)/(n + \alpha + \beta - 2)$; Laplace smoothing ($\alpha = \beta = 2$ style "add-one") is the classic instance, e.g. $(3 + 1)/(3 + 2) = 0.8$. This is exactly why Naive Bayes uses smoothing.

Q27. Explain, using this chapter's tools, why "we tested 25 metrics and shipped because engagement was significant at $p = 0.04$ " is a mistake.

Multiple comparisons: under 25 true nulls at $\alpha = 0.05$, the expected number of false positives is 1.25 — a single $p = 0.04$ among 25 metrics is entirely consistent with no real effect anywhere ($P(\text{at least one sig}) = 1 - 0.95^{25} \approx 72\%$). Fixes: pre-register one primary metric; correct thresholds (Bonferroni α/m , or Benjamini-Hochberg FDR for many metrics); treat the others as exploratory, to be confirmed in a follow-up test.

Q28. What is mutual information, and when would you prefer it to correlation for feature selection?

$I(X; Y) = H(Y) - H(Y | X)$: how many bits knowing X removes from the uncertainty of Y ; zero iff independent. Prefer it when relationships may be nonlinear or non-monotonic (correlation misses them), or for categorical variables where Pearson doesn't apply. Costs: it needs density/histogram estimation (harder with little data), returns no direction/sign, and like all filter methods scores features individually — it can miss features useful only in combination.

Chapter 2: Python for ML Interviews

Python questions in an ML interview are rarely about Python for its own sake. They are a proxy for a sharper question: *can this person turn an idea into working, efficient, maintainable code?* A candidate who reaches for a generator when data doesn't fit in memory, who knows why `apply(axis=1)` is a thousand times slower than a vectorized expression, and whose data pipeline is testable rather than a 400-line script, is someone who can be trusted with production ML. That is what the interviewer is buying.

This chapter covers the Python that ML interviews actually test: the core language features that come up in screens (data structures, comprehensions, generators, decorators, OOP), the two libraries every ML role assumes (NumPy and Pandas) at the level where performance questions live, the complexity of common operations, and the habits that make ML code clean and testable. Algorithmic problem-solving — the LeetCode-style material — is Chapter 3; this chapter is about *fluency*.

Core Python

Built-in data structures: what they are and what they cost

Python's four workhorse containers each have a specific implementation, and interviewers expect you to know the cost model that follows from it.

A **list** is a dynamic array: a contiguous block of pointers to objects, over-allocated so that appending is cheap. Indexing and `append` are $O(1)$ (`append` is *amortized* $O(1)$ — occasionally the array is reallocated and copied), but inserting or deleting at the front is $O(n)$ because every element shifts, and `x in my_list` is $O(n)$ because there is nothing to do but scan. Lists are mutable and ordered; they are the default sequence.

A **tuple** is an immutable list. Immutability buys three things: tuples can be dictionary keys and set members (they are hashable, provided their contents are); they are slightly smaller and faster to create; and they communicate intent — a `(latitude, longitude)` pair is a fixed-shape record, not a growable collection. Returning multiple values from a function is tuple packing; `lat, lon = get_coords()` is unpacking.

A **dict** is a hash table: keys are hashed to slots, giving $O(1)$ average insertion, lookup, and deletion. Since Python 3.7, dicts preserve insertion order as a language guarantee. The cost of hashing is worst-case degradation: adversarial or terrible hash functions make operations $O(n)$, which is a classic follow-up question. In ML, dicts are everywhere the moment data stops being rectangular: vocabularies mapping token $\rightarrow$ id, configuration objects, feature registries, label maps.

A **set** is a dict with only keys: $O(1)$ average membership testing, plus set algebra (union, intersection, difference). The two ML reflexes: deduplication (`set(user_ids)`) and fast membership (`if token in stopwords` — with a list of 500 stopwords this is the difference between $O(1)$ and $O(n)$ *per token*). Sets are unordered and their elements must be hashable.

Worked example. You process 1M tokens against 500 stopwords. With a *stopword list*: $1M \times O(500)$ scans $\approx 5 \times 10^8$ comparisons. With a *set*: $1M \times O(1) \approx 10^6$ hashes — a $\sim 500\times$ improvement from a one-character change. This is the most common "spot the inefficiency" bait in ML code screens.

Comprehensions: transformation as expression

A comprehension builds a container from an iterable in one readable expression:

```
squares = [x**2 for x in range(10) if x % 2 == 0] # list
vocab    = {tok: i for i, tok in enumerate(tokens)} # dict
uniq_len = {len(w) for w in words}                # set
lazy     = (x**2 for x in range(10**9))           # generator expression - no memory
```

Comprehensions are preferred over `map/filter` with lambdas in modern Python style, and are faster than an explicit `for` loop building a list with `.append` (the loop machinery is pushed into C). Two rules of taste that interviewers notice: don't nest more than two levels (extract a function instead), and don't use a comprehension *for its side effects* — a comprehension should build a value, not call `print` or mutate state.

The fourth form above — parentheses instead of brackets — is not a "tuple comprehension"; it is a **generator expression**, which brings us to the most important core-language topic for ML.

Generators and iterators: computing lazily

An **iterator** is any object that produces values one at a time via `__next__` and raises `StopIteration` when done; an **iterable** is anything that can produce an iterator via `__iter__`. The `for` loop is sugar over this protocol. A **generator** is the easy way to write an iterator: any function containing `yield` returns, when called, a paused computation that produces values on demand.

```
def read_records(path):
    with open(path) as f:
        for line in f:           # file objects are themselves lazy iterators
            yield parse(line)    # one record in memory at a time
```

Why ML interviews care: **training data usually does not fit in memory**. A generator-based pipeline — read a record, transform it, yield it, batch it — processes a 500 GB dataset with a constant memory footprint. This is precisely the design of PyTorch's `IterableDataset`, TensorFlow's `tf.data`, and every streaming feature pipeline. When an

interviewer asks "how would you train on data larger than RAM?", the first word of the answer is "stream": generators are the language-level mechanism.

The properties to state cold: generators are **lazy** (nothing computes until requested), **single-use** (once exhausted, they are empty — iterating twice is a real bug the code section demonstrates), and **cheap** (a generator object is ~100 bytes regardless of what it will produce). `yield` also supports pipelining: `sum(x**2 for x in stream)` composes without materializing anything. The `itertools` module (`islice`, `chain`, `cycle`, `groupby`) is the standard toolkit for combining them.

Decorators: functions that wrap functions

Python functions are first-class objects: they can be passed, returned, and stored. A **closure** is a function that captures variables from its enclosing scope. A **decorator** is a function that takes a function and returns a wrapped version — and `@decorator` is syntax sugar for `fn = decorator(fn)`.

```
def retry(times):
    def decorator(fn):
        @functools.wraps(fn)           # keep fn's name and docstring
        def wrapper(*args, **kwargs):
            for attempt in range(times):
                try:
                    return fn(*args, **kwargs)
                except TransientError:
                    continue
            raise
        return wrapper
    return decorator

@retry(times=3)
def call_feature_store(key): ...
```

The `*args`, `**kwargs` idiom makes the wrapper signature-agnostic: `*args` collects positional arguments into a tuple, `**kwargs` collects keyword arguments into a dict, and the wrapper forwards both untouched. `functools.wraps` copies the wrapped function's metadata onto the wrapper — omit it and every decorated function reports its name as `wrapper`, which breaks debugging and serialization.

Where decorators appear in ML systems: timing and logging (`@timed`), caching (`functools.lru_cache` — memoization in one line), retries around flaky I/O, input validation, registration (frameworks collect models or routes via `@register("resnet50")`), and PyTorch's `@torch.no_grad()` which switches off gradient tracking around evaluation code. Interviewers ask candidates to *write* a timing decorator often enough that the code section includes one.

OOP for ML: the sklearn pattern

ML interviews test OOP less as trivia ("what is polymorphism?") and more as design: *can you structure model code the way real libraries do?* The pattern to internalize is scikit-

learn's **estimator API**: a class with `fit(X, y)` that learns state (conventionally stored with a trailing underscore: `self.mean_`), `transform(X)` or `predict(X)` that applies it, and nothing else happening in `__init__` except storing hyperparameters. The power of the convention is composability — a `Pipeline` can chain any objects that follow it, and cross-validation can clone and refit them. The code section implements a `StandardScaler` this way, with tests.

Language mechanics worth having ready:

Dunder (double-underscore) methods define how objects respond to syntax. `__init__` constructs; `__repr__` displays; `__call__` makes an instance callable — the reason a PyTorch `model(x)` works; `__len__` and `__getitem__` make an object indexable and iterable — exactly the protocol of a PyTorch map-style `Dataset`, which is nothing more than a class defining those two methods.

Inheritance vs composition. Inherit when the subclass genuinely *is* a specialization sharing the parent's interface (`class ResNet(nn.Module)`); compose when a thing *has* a part (`Pipeline` has steps; a `Trainer` has a model, an optimizer, a scheduler). The interview-safe rule: prefer composition, use inheritance for framework base classes designed for it. Deep inheritance trees in ML code are a smell; two levels is usually plenty.

@staticmethod vs @classmethod vs instance methods. Instance methods receive `self`; classmethods receive `cls` and are the standard idiom for alternate constructors (`Model.from_pretrained(...)`, `Config.from_yaml(path)`); staticmethods receive neither and are namespaced utility functions.

Dataclasses. `@dataclass` autogenerates `__init__`, `__repr__`, and `__eq__` from field declarations — the modern way to write configuration objects (`TrainingConfig(lr=3e-4, batch_size=32, epochs=10)`) instead of dicts, gaining attribute access, defaults, and type hints that IDEs and reviewers can check.

NumPy

The ndarray: why it is fast

A Python list of a million floats is a million separate heap objects plus a million pointers; every arithmetic operation dispatches dynamically, object by object. A NumPy **ndarray** is one contiguous block of raw memory with a single **dtype** (say, `float64`), described by a **shape** and **strides** (how many bytes to step per dimension). Operations on it run as compiled C loops over machine numbers — no per-element interpretation, cache-friendly memory access, and SIMD where available. That design gap, not magic, is the 10-100× speedups this section measures.

Two consequences follow directly from the memory model and generate a disproportionate share of interview questions:

Views vs copies. Because an array is (shape, strides, pointer), many operations can return a new *view* of the same buffer instead of copying: basic slicing (`a[:, 1:3]`), `reshape` (usually), and `transpose` all do. Mutating a view mutates the original. **Fancy indexing** — indexing with an integer array or boolean mask — must gather scattered elements, so it always returns a *copy*. Knowing which is which prevents both accidental-mutation bugs and accidental-copy performance cliffs; the code section demonstrates both with `np.shares_memory`.

dtype discipline. `float64` is the default, but half the memory (`float32`) is the working standard in deep learning, and integer overflow is silent in fixed-width dtypes (`np.array([200], dtype=np.int8) + 100` wraps around to a negative number, with no exception). When a pipeline mysteriously doubles its memory, a stray `float64` upcast is the first suspect.

Vectorization: replace the loop with the array

Vectorization means expressing a computation as whole-array operations so the loop runs in C rather than in the interpreter. The interview rule: *if you wrote `for` over array elements, you probably made a mistake.*

The canonical transformation, measured in the code section: applying a sigmoid to a million values with a Python-level loop takes ~0.5 s; the expression `1.0 / (1.0 + np.exp(-z))` takes ~9 ms — a **54× speedup** for identical results. Everything in the expression — negation, `exp`, add, divide — is a **ufunc** (universal function) that maps over arrays elementwise in compiled code.

Vectorized thinking is a *translation skill*: conditional logic becomes `np.where(cond, a, b)`; accumulation becomes `np.cumsum`; "does any/all row satisfy X" becomes `mask.any(axis=1)`; counting becomes `(y_pred == y_true).mean()` (accuracy in one expression, exploiting the fact that booleans are 0/1). The reductions all take an **axis** argument, and axis semantics are a favorite quick check: for a matrix, `axis=0` collapses rows (result: one value per column), `axis=1` collapses columns (one value per row). "The axis you name is the axis that disappears" is the reliable mnemonic.

Broadcasting: arithmetic across mismatched shapes

Broadcasting is the rule NumPy uses to combine arrays of different shapes without copying data. Align the shapes *from the right*; two dimensions are compatible if they are equal, or if either is 1 (that dimension is conceptually stretched to match); missing leading dimensions are treated as 1.

X	(1000, 64)		X	(1000, 1, 64)
mu	(64)	-> OK	C	(1, 10, 64) -> result (1000, 10, 64)

The left example is the everyday case: `X - X.mean(axis=0)` centers every column, the (64,) mean broadcasting across 1000 rows. The right example is the power move: insert size-1

axes with `None/np.newaxis` so that *all pairs* interact — `X[:, None, :] - C[None, :, :]` computes every point-minus-centroid difference in one expression, giving pairwise distances for k-means with no loop (code section, Listing 3).

Two hazards to volunteer in an interview. **Silent shape bugs:** subtracting a `(n,)` vector from a `(n, 1)` column doesn't error — it broadcasts to `(n, n)`, producing a plausibly-shaped wrong answer downstream. This is the argument for keeping vectors deliberately 1-D or deliberately 2-D and for `assert`-ing shapes at function boundaries. **Memory blowup:** broadcasting avoids copying *inputs*, but the *output* is fully materialized — the `(1000, 10, 64)` temporary above is fine, but the same pattern with 100k points and 10k centroids would try to allocate hundreds of GB. The fix is chunking, or algebra: expand the squared distance into norms and a matrix product ($||x-c||^2 = ||x||^2 + ||c||^2 - 2x \cdot c$), which needs only a `(n, k)` result.

Array operations you must know

Beyond arithmetic, a short list of operations covers most interview code. `reshape(-1, d)` (the -1 means "infer this dimension"); `argmax/argmin` and `argsort` (indices, not values — `np.argsort(scores)[::-1][:k]` is top-k); boolean masking (`X[y == 1]` selects the positive class); `np.unique(labels, return_counts=True)` (class distribution in one call); `concatenate/stack` (join on an existing axis vs a new one); and `np.random.default_rng(seed)` — the modern, explicitly-seeded random API, which matters because reproducibility questions ("your results differ between runs — why?") are stock interview material. For linear algebra: `@` (matrix multiply), `np.linalg.solve` (never invert explicitly — Chapter 1), and `np.einsum` for anything with more than two indices (worth recognizing even if you don't write it fluently: `np.einsum('bik,bkj->bij', A, B)` is a batched matrix multiply, and attention implementations read this way).

Pandas

The DataFrame model

A **Series** is a 1-D array with an **index** (labels); a **DataFrame** is a dict of Series sharing one index — columns each have a dtype, rows are labeled. The index drives pandas' signature behavior, **alignment**: operations between Series match on labels, not positions, so adding two revenue Series from different months aligns by date and produces NaN where either is missing. Alignment is why pandas feels magical when indexes agree and baffling when they don't; `reset_index()` is the escape hatch back to plain positional data.

Two selection idioms must be automatic: `df.loc[rows, cols]` selects by *label* (inclusive slices), `df.iloc[rows, cols]` by *position* (exclusive slices, like Python). Mixing them up — or chaining selections like `df[df.x > 0]['y'] = 1`, which may modify a temporary copy instead of `df` and triggers the infamous `SettingWithCopyWarning` — is the most common pandas bug class; the fix is always a single `.loc`: `df.loc[df.x > 0, 'y'] = 1`.

groupby: split-apply-combine

`groupby` implements the **split-apply-combine** pattern: partition rows by key, apply a computation per group, stitch the results together. The interview-relevant distinctions are between the three apply-stage methods:

agg reduces each group to one row per statistic —

`df.groupby('user_id').agg(revenue=('amount', 'sum'), n=('amount', 'size'))` produces per-user aggregates (and named aggregation, shown here, keeps columns tidy). **transform** returns a result *the same length as the input*, broadcast back to every row of the group — which is exactly what feature engineering needs: `df['amt_vs_user_mean'] = df.amount / df.groupby('user_id').amount.transform('mean')` attaches "this transaction relative to the user's average" to every transaction, no merge required. **apply** runs an arbitrary function per group — maximally flexible, minimally fast, the last resort.

`size` vs `count` is a small trap: `size` counts rows, `count` counts *non-null* values.

merge and join: SQL in memory

`pd.merge(left, right, on='key', how=...)` is a hash join. The four `how` modes match SQL: `inner` keeps matched keys only, `left` keeps all left rows (unmatched right columns become NaN), `right` mirrors it, `outer` keeps everything. The code section's pipeline shows the practical read: after a left join of per-user aggregates onto a segments table, a user present in events but absent from segments survives with `segment = NaN` — and a user with no events silently disappears, which is precisely the kind of row-count change to check after every merge (`len(result)` vs expectation, or `validate='one_to_one' / 'many_to_one'` to make the assumption explicit and loudly enforced).

The failure mode interviewers probe: merging on a key that is non-unique on *both* sides produces the Cartesian product of matches — a 10-row × 10-row duplicate key becomes 100 rows, and at scale this is the "my join exploded to 40 GB" incident. State the guard: know which side is supposed to be unique, `drop_duplicates` first, and use `validate`.

pivot and reshape: wide vs long

`pivot_table(index=..., columns=..., values=..., aggfunc=...)` turns long data (one row per observation) into wide data (one row per entity, one column per category), aggregating duplicates — the code section builds a segment × event-type count table in one call. Its inverse is `melt`, wide back to long. The ML relevance: models want wide (one row per example), storage and event logs are long (one row per event), so real feature pipelines pivot constantly. `pd.crosstab(y_true, y_pred)` — a special-cased pivot — is a confusion matrix in one line.

Missing data

Pandas historically represents missingness with the float `NaN`, which imposes a gotcha: an integer column acquiring a `NaN` silently becomes float (nullable dtypes like `Int64` fix this, at the cost of remembering they exist). The mechanics: `isna()` to detect (never `== np.nan`, which is always `False` — `NaN` compares unequal to everything including itself), `dropna()` to remove, `fillna(value)` to impute, and `NaN` is contagious through arithmetic while being *skipped by default* in reductions (`mean(skipna=True)`) and *dropped by default* by groupby keys — both defaults can quietly bias an analysis.

The ML-critical points: first, **imputation is a model decision, not a cleanup chore** — filling with the median changes the distribution, filling with 0 invents a signal (the code section fills view-events' amounts with 0 because "views have no purchase amount" is *structural* missingness, a semantically different case from "amount lost in logging"); often the strongest move is adding a `was_missing` indicator column and letting the model use it. Second, **impute with statistics computed on the training set only** — computing the median over all data before splitting is leakage (Chapter 9 treats this fully).

apply vs vectorized operations

`df.apply(fn, axis=1)` calls a Python function once per row, constructing a Series for each — it is a Python loop wearing a pandas costume. The code section measures the same multiplication done three ways; the vectorized column expression is **~2,000× faster** than row-wise `apply` on a million rows. The escalation ladder when you're tempted to `apply`: (1) a vectorized column expression (`df.price * df.qty`); (2) `np.where` / `np.select` for conditionals; (3) `.map(dict)` for lookups; (4) `.str` / `.dt` accessor methods for text and dates; (5) only then `apply`, and prefer column-wise (`axis=0`) over row-wise. If genuinely stuck with per-row logic at scale, drop to NumPy arrays or vectorize the function itself.

Time and space complexity of common operations

Interviewers rarely ask for formal proofs here; they ask "what's the cost of X?" mid-discussion and listen for instant, correct answers. The table below is the required inventory. (Big-O review and algorithm design live in Chapter 3.)

Operation	Average complexity	Notes
<code>lst[i]</code> , <code>lst.append(x)</code>	$O(1)$	append amortized (occasional realloc + copy)
<code>lst.insert(0, x)</code> , <code>lst.pop(0)</code>	$O(n)$	shifts everything; use <code>collections.deque</code> for $O(1)$ ends
<code>x in lst</code>	$O(n)$	linear scan
<code>x in set_</code> / <code>d[key]</code>	$O(1)$	hash table; $O(n)$ worst case (collisions)

Operation	Average complexity	Notes
<code>sorted(lst), lst.sort()</code>	$O(n \log n)$	Timsort; $O(n)$ on nearly-sorted data; stable
<code>heapq push/pop</code>	$O(\log n)$	top-k pattern: heap of size $k \rightarrow O(n \log k)$
string concat in a loop	$O(n^2)$ total	strings immutable $\rightarrow$ each <code>+=</code> copies; use <code>''.join(parts)</code>
slice <code>lst[a:b]</code>	$O(b-a)$	copies (list); NumPy slicing is $O(1)$ — returns a view
<code>np.dot(A, B), A @ B</code>	$O(m \cdot n \cdot p)$	$(m \times n)(n \times p)$; BLAS constant factors are tiny
elementwise ufunc on n elements	$O(n)$	C-speed; the constant is $\sim 100\times$ smaller than a Python loop
<code>np.argsort</code>	$O(n \log n)$	<code>np.argpartition</code> gives unordered top-k in $O(n)$
<code>pd.merge</code> (hash join)	$\sim O(n+m)$	plus output size — which is $O(n \cdot m)$ if keys duplicate on both sides!
<code>groupby().agg()</code>	$\sim O(n)$	hash-based grouping

Space costs worth stating: a Python float is ~ 24 bytes of object overhead plus pointer versus 8 bytes in a `float64` array (and 4 in `float32`) — the reason "load it into NumPy" is itself a memory optimization; list over-allocation runs $\sim 12\%$; a dict costs roughly $2\text{--}4\times$ the raw key/value memory in table overhead; and every pandas operation that returns a new DataFrame (most do) briefly holds two copies, which is why memory-bound pipelines chunk their work or move to generators.

The top-k idiom deserves its sixty seconds: sorting all n scores to take k is $O(n \log n)$; a size- k heap is $O(n \log k)$; `np.argpartition(scores, -k)` is $O(n)$. Recommending the right one for "find the 10 nearest neighbors among 10M vectors" is a stock systems-flavored coding question.

Writing clean, testable ML code

ML code rots in specific, predictable ways — hidden global state, irreproducible randomness, leaky preprocessing, and notebooks that became load-bearing. Interviewers increasingly test for the antibodies directly ("how would you test this pipeline?"), and strong answers are concrete:

Structure: config $\rightarrow$ data $\rightarrow$ model $\rightarrow$ train $\rightarrow$ evaluate. Separate these stages into functions or small classes with explicit inputs and outputs; keep hyperparameters in one dataclass rather than scattered literals; make every function referentially honest (same inputs $\rightarrow$ same outputs) except at clearly marked randomness boundaries. The sklearn estimator convention (hyperparameters in `__init__`, learned state as `attr_` set by `fit`, no I/O inside the class) is the pattern to name.

Reproducibility. Seed everything explicitly (`np.random.default_rng(seed)` passed in, not global `np.random.seed` sprinkled around), pin dependency versions, log the config with every result. "Your metric changed and no code changed — what happened?" is a stock question; unseeded shuffling, dict-ordering assumptions in old versions, and nondeterministic GPU kernels are the expected suspects.

Tests for ML code differ from generic unit tests, and naming the categories is a strong signal. *Shape and dtype tests*: the transform outputs (n, d) float32, no NaNs. *Statistical property tests*: standardized output has mean ≈ 0 , std ≈ 1 (Listing 7 does exactly this). *Contract tests*: transform before fit raises; constant column doesn't crash. *Leakage tests*: transforming a held-out point uses training statistics (Listing 7's `test_no_leakage_train_stats_reused` — arguably the highest-value five-line test in ML). *Overfit-one-batch test*: a working model driven to near-zero loss on 10 examples; if it can't, the wiring is broken — the fastest smoke test in deep learning. *Golden/regression tests*: pin a small end-to-end run's metric within a tolerance.

Fail loudly at boundaries. `assert X.ndim == 2`, validate merges, raise on unexpected NaN influx. Silent shape coercion and silent NaN propagation are how ML bugs survive to production; a one-line assert converts a week of debugging into a stack trace.

Code implementations

Every listing below was executed as printed; outputs are real.

Listing 1 — Generators: constant memory and single-use semantics

```
import sys

# Eager: materialize a million squared numbers
eager = [i * i for i in range(1_000_000)]
# Lazy: a recipe that produces them one at a time
lazy = (i * i for i in range(1_000_000))

print(f"list size: {sys.getsizeof(eager)/1e6:.1f} MB")
print(f"generator size: {sys.getsizeof(lazy)} bytes")

def batch_iter(data, batch_size):
    """Yield successive batches without copying the dataset."""
    for start in range(0, len(data), batch_size):
        yield data[start:start + batch_size] # yield pauses here, resumes on next()

batches = batch_iter(list(range(10)), batch_size=4)
print("batches:", [b for b in batches])
print("exhausted:", [b for b in batches]) # generators are single-use!
```

Output:

```
list size: 8.4 MB
generator size: 104 bytes
batches: [[0, 1, 2, 3], [4, 5, 6, 7], [8, 9]]
exhausted: []
```


An 8.4 MB list versus a 104-byte generator *for the same computation* — that gap is why data loaders stream. `batch_iter` is a five-line version of what every ML framework's loader does (note it yields a final short batch — decide deliberately whether to keep or drop it). The last line is the classic bug: the second iteration silently produces nothing, which in real code looks like "my second epoch trained on zero batches."

Listing 2 — Decorators: timing and memoization

```
import functools, time

def timed(fn):
    """Decorator: log how long fn takes, without touching its code."""
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        result = fn(*args, **kwargs)
        print(f"{fn.__name__}: {time.perf_counter() - t0:.4f}s")
        return result
    return wrapper

def memoize(fn):
    """Cache results by argument - dynamic programming for free."""
    cache = {}
    @functools.wraps(fn)
    def wrapper(*args):
        if args not in cache:
            cache[args] = fn(*args)
        return cache[args]
    return wrapper

@memoize
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

@timed
def run():
    return fib(300)

print("fib(300) has", len(str(run())), "digits")
```

Output:

```
run: 0.0010s
fib(300) has 63 digits
```

Without memoization, naive `fib(300)` would need $\sim 2^{300}$ calls — the memoized version finishes in a millisecond because each subproblem is computed once and cached in the closure's dict. `memoize` is a hand-rolled `functools.lru_cache` (use the built-in in real code; write it by hand in interviews). Note the two-layer structure of `timed`: the outer function receives `fn`, the inner `wrapper` receives the call arguments, and the closure connects them.

Listing 3 — Vectorization and broadcasting

```
import numpy as np, time
rng = np.random.default_rng(42)
z = rng.normal(size=1_000_000)

t0 = time.perf_counter()
out_loop = [1.0 / (1.0 + np.exp(-v)) for v in z]      # Python-level loop
t_loop = time.perf_counter() - t0

t0 = time.perf_counter()
out_vec = 1.0 / (1.0 + np.exp(-z))                  # one C-level pass
t_vec = time.perf_counter() - t0

print(f"loop: {t_loop:.2f}s | vectorized: {t_vec*1000:.0f} ms | "
      f"speedup: {t_loop/t_vec:.0f}x")
print("identical:", np.allclose(out_loop, out_vec))

# Broadcasting: pairwise distances between 1000 points and 10 centroids
X = rng.normal(size=(1000, 64)); C = rng.normal(size=(10, 64))
# (1000, 1, 64) - (1, 10, 64) -> broadcasts to (1000, 10, 64)
D = np.sqrt(((X[:, None, :] - C[None, :, :]) ** 2).sum(axis=2))
print("distance matrix shape:", D.shape,
      "| nearest centroid of first 3 points:", D[:3].argmin(axis=1))
```

Output:

```
loop: 0.47s | vectorized: 9 ms | speedup: 54x
identical: True
distance matrix shape: (1000, 10) | nearest centroid of first 3 points: [4 5 7]
```

The sigmoid comparison is the whole vectorization argument in six lines: same math, same results, 54× apart. The distance computation is the assignment step of k-means (Chapter 8) with zero loops: two `None`-inserted axes make the shapes broadcast-compatible, the subtraction materializes all 1000×10 difference vectors, and the axis-2 reduction collapses them to distances.

Listing 4 — Views vs copies

```
import numpy as np
a = np.arange(12).reshape(3, 4)
s = a[:, 1:3]      # slicing returns a VIEW - shares memory
s[0, 0] = 99      # mutates a as well
print("a[0] after editing the slice:", a[0])
f = a[a > 5]      # boolean (fancy) indexing returns a COPY
f[0] = -1
print("a unchanged by editing the copy:", a[1])
print("view shares memory:", np.shares_memory(a, s),
      "| fancy copy shares:", np.shares_memory(a, f))
```

Output:

```
a[0] after editing the slice: [ 0 99  2  3]
a unchanged by editing the copy: [4 5 6 7]
view shares memory: True | fancy copy shares: False
```

Writing 99 into the slice changed the original array — that's a view doing exactly what views do. When you need an independent slice, say `.copy()` explicitly. `np.shares_memory` is the diagnostic to name when asked "how would you check?"

Listing 5 — A pandas pipeline: missing data, groupby, merge, pivot

```
import numpy as np
import pandas as pd

events = pd.DataFrame({
    "user_id": [1, 1, 2, 2, 2, 3, 4, 4],
    "event":    ["view", "buy", "view", "view", "buy", "view", "buy", "buy"],
    "amount":   [np.nan, 20.0, np.nan, np.nan, 35.0, np.nan, 15.0, np.nan],
})
users = pd.DataFrame({
    "user_id": [1, 2, 3, 5],
    "segment": ["free", "pro", "free", "pro"],
})

# 1. Missing data: views have no amount - structural, so fill with 0
print("missing amounts:", events["amount"].isna().sum())
events["amount"] = events["amount"].fillna(0.0)

# 2. groupby: split-apply-combine
per_user = (events.groupby("user_id")
            .agg(n_events=("event", "size"),
                 n_buys=("event", lambda s: (s == "buy").sum()),
                 revenue=("amount", "sum"))
            .reset_index())

print(per_user)

# 3. merge: LEFT join keeps user 4 (no segment); user 5 (no events) drops
full = per_user.merge(users, on="user_id", how="left")
print("segment NaN for unmatched user:", full.loc[full.user_id == 4,
"segment"].isna().item())

# 4. pivot table: segment x event counts
pivot = events.merge(users, on="user_id", how="inner").pivot_table(
    index="segment", columns="event", values="user_id", aggfunc="count", fill_value=0)
print(pivot)
```

Output:

```
missing amounts: 5
  user_id  n_events  n_buys  revenue
0        1         2        1     20.0
1        2         3        1     35.0
2        3         1        0      0.0
3        4         2        2     15.0
segment NaN for unmatched user: True
event    buy  view
segment
free      1    2
pro       1    2
```

Eight rows of events and four users exercise the full toolkit. Follow one user through: user 4 has two buys totaling 15.0 (one buy amount was missing → filled to 0 — a judgment call the comment documents), survives the left join with a NaN segment, and is *absent* from the pivot because the pivot's inner merge drops users without segments.

Every row-count change here is intentional and checked; in production the same three operations run on millions of rows, and the discipline is identical.

Listing 6 — apply vs vectorized: the 2,000× gap

```
import numpy as np, pandas as pd, time
rng = np.random.default_rng(0)
df = pd.DataFrame({"price": rng.uniform(1, 100, 1_000_000),
                  "qty": rng.integers(1, 10, 1_000_000)})

t0 = time.perf_counter()
r1 = df.apply(lambda row: row["price"] * row["qty"], axis=1)  # row-wise Python loop
t_apply = time.perf_counter() - t0

t0 = time.perf_counter()
r2 = df["price"] * df["qty"]  # vectorized
t_vec = time.perf_counter() - t0

print(f"apply(axis=1): {t_apply:.2f}s | vectorized: {t_vec*1000:.1f} ms | "
      f"speedup: {t_apply/t_vec:.0f}x")
print("identical:", np.allclose(r1, r2))
```

Output:

```
apply(axis=1): 5.64s | vectorized: 2.6 ms | speedup: 2188x
identical: True
```

Not a typo: **two thousand times slower** for the same multiplication. `apply(axis=1)` constructs a Series object per row and crosses the Python/C boundary a million times; the column expression crosses it once. If a candidate remembers one pandas fact from this chapter, this is the one.

Listing 7 — A testable transformer, sklearn-style

```
import numpy as np

class StandardScaler:
    """Minimal sklearn-style transformer: fit on train, transform anywhere."""

    def fit(self, X):
        X = np.asarray(X, dtype=float)
        self.mean_ = X.mean(axis=0)
        self.scale_ = X.std(axis=0)
        self.scale_[self.scale_ == 0.0] = 1.0  # constant column: avoid 0-division
        return self

    def transform(self, X):
        if not hasattr(self, "mean_"):
            raise RuntimeError("call fit() before transform()")
        return (np.asarray(X, dtype=float) - self.mean_) / self.scale_

    def fit_transform(self, X):
        return self.fit(X).transform(X)

# ---- tests: small, deterministic, one behavior each ----
def test_output_is_standardized():
    rng = np.random.default_rng(0)
```

```

Z = StandardScaler().fit_transform(rng.normal(5, 3, size=(200, 4)))
assert np.allclose(Z.mean(axis=0), 0, atol=1e-9)
assert np.allclose(Z.std(axis=0), 1, atol=1e-9)

def test_no_leakage_train_stats_reused():
    scaler = StandardScaler().fit([[0.0], [10.0]]) # mean 5, std 5
    assert np.allclose(scaler.transform([[15.0]]), [[2.0]]) # uses TRAIN stats

def test_constant_column_does_not_crash():
    Z = StandardScaler().fit_transform([[3.0], [3.0], [3.0]])
    assert np.allclose(Z, 0.0)

def test_transform_before_fit_raises():
    try:
        StandardScaler().transform([[1.0]])
        assert False, "should have raised"
    except RuntimeError:
        pass

for t in [test_output_is_standardized, test_no_leakage_train_stats_reused,
         test_constant_column_does_not_crash, test_transform_before_fit_raises]:
    t(); print(f"PASS {t.__name__}")

```

Output:

```

PASS test_output_is_standardized
PASS test_no_leakage_train_stats_reused
PASS test_constant_column_does_not_crash
PASS test_transform_before_fit_raises

```

Twenty lines of class, four tests, and each test encodes a real failure mode: statistical correctness, leakage (the transform of a new point must use *training* statistics — the test would catch a refit-on-transform bug immediately), the divide-by-zero edge case, and the usage contract. In a real project these run under `pytest`; the pattern — tiny deterministic inputs, one behavior per test, assertions on properties rather than exact matrices — is what "how would you test ML code?" wants to hear.

Listing 8 — The mutable default argument

```

def add_feature(row, features=[]): # BUG: default evaluated once, shared forever
    features.append(row)
    return features

print(add_feature("age"))
print(add_feature("income")) # surprise: 'age' is still there

def add_feature_fixed(row, features=None):
    features = [] if features is None else features
    features.append(row)
    return features

print(add_feature_fixed("age"))
print(add_feature_fixed("income"))

```

Output:

```

['age']
['age', 'income']

```

```
[ 'age' ]
[ 'income' ]
```

Default argument values are evaluated **once, at function definition** — every call without the argument shares the same list, which accumulates state across calls. This is the single most-asked Python gotcha in interviews, and it bites real ML code (feature lists, config dicts as defaults). The `None` sentinel is the standard fix.

Pitfalls, comparisons and practical tips

Confusion	Resolution
list vs tuple	Mutable vs immutable. Tuples are hashable (usable as dict keys/set members), slightly cheaper, and signal fixed structure.
<code>is</code> vs <code>==</code>	<code>is</code> compares identity (same object), <code>==</code> compares value. Only use <code>is</code> for <code>None</code> . Small-int caching makes <code>a is b</code> "work" for 5 and fail for 500 — never rely on it.
shallow vs deep copy	<code>list(x)</code> , <code>x.copy()</code> , <code>copy.copy</code> duplicate the container but share nested objects; <code>copy.deepcopy</code> recurses. Nested-config mutation bugs live here.
view vs copy (NumPy)	Slices/reshape/transpose → views (mutations propagate); fancy/boolean indexing → copies. Check with <code>np.shares_memory</code> ; force with <code>.copy()</code> .
<code>(n,)</code> vs <code>(n,1)</code>	1-D vs 2-D column. Mixing them broadcasts to <code>(n,n)</code> silently. Assert shapes at function boundaries.
<code>axis=0</code> vs <code>axis=1</code>	The axis you name is the axis that disappears: <code>axis=0</code> collapses rows → per-column result; <code>axis=1</code> collapses columns → per-row result.
<code>loc</code> vs <code>iloc</code>	Label-based (inclusive slices) vs position-based (exclusive). Chained indexing + assignment → <code>SettingWithCopyWarning</code> ; always assign through a single <code>.loc</code> .
<code>agg</code> vs <code>transform</code> vs <code>apply</code> (groupby)	One row per group vs same-length broadcast (feature engineering) vs arbitrary-and-slow.
<code>size</code> vs <code>count</code>	Rows vs non-null values.
<code>NaN == NaN</code>	Always False. Use <code>isna()</code> . <code>NaN</code> also coerces int columns to float.
merge row-count changes	Left join preserves left row count only if the right key is unique — duplicate keys multiply rows. Use <code>validate=</code> , check <code>len()</code> after.
mutable default argument	Evaluated once at def-time and shared across calls. Sentinel: <code>def f(x, acc=None)</code> .
generator reuse	Generators are single-use; a second loop over one yields nothing. Re-create it or materialize a list deliberately.
string <code>+=</code> in a loop	$O(n^2)$. Collect in a list, <code>''.join(parts)</code> .

Confusion	Resolution
the GIL	One thread executes Python bytecode at a time → threads don't speed up pure-Python CPU work. NumPy/pandas release the GIL in C sections; use <code>multiprocessing</code> (or <code>vectorize</code>) for CPU-bound Python.

Practical tips for the live-coding portion: narrate your data-structure choices as you make them ("set, because I need membership tests") — the choice *is* the answer being graded. When writing NumPy in front of an interviewer, say shapes out loud at every step; shape errors caught verbally are free, shape errors caught by traceback cost minutes. And when asked to speed something up, profile before optimizing — `%timeit` on the suspect line, not guesswork — then reach for vectorization first, algorithmic improvements second, parallelism last.

Interview questions and answers

Core Python

Q1. When would you choose a tuple over a list, and why can tuples be dict keys when lists can't?

Choose a tuple for fixed-shape records (coordinates, RGB values, function multi-returns), when you need hashability, or to signal immutability. Dict keys must be hashable — the hash must stay constant for the table to find the key again. A list can mutate after insertion, changing what its hash *would* be, so lists are unhashable by design; a tuple's contents can't be reassigned (note: a tuple containing a list is still unhashable — hashability is recursive).

Q2. How does a Python dict achieve $O(1)$ lookup, and when does that guarantee fail?

Keys are hashed to an index in a sparse table; lookup hashes the query, jumps to the slot, and resolves collisions by open addressing (probing). Average $O(1)$ assumes hashes spread keys evenly. It degrades toward $O(n)$ when many keys collide — pathological or adversarial hash inputs (the reason Python randomizes string hashing per process) — and every dict operation pays a rehash/resize occasionally as the table grows. *Interviewers listen for: hash → slot → collision handling, plus one failure mode.*

Q3. What's the difference between an iterable, an iterator, and a generator?

An iterable can produce an iterator (`__iter__`) — lists, dicts, files. An iterator produces values via `__next__` and raises `StopIteration` when exhausted; it's the thing a for loop actually drives. A generator is an iterator created by a function containing `yield` (or a `genexp`): the function body runs lazily, pausing at each `yield`. All generators are iterators; all iterators are iterable; a list is iterable but not an iterator (you can loop over it repeatedly).

Q4. Write (or describe) a decorator that caches a function's results. What are the limitations?

Closure over a dict keyed by args: if `args not in cache`, compute and store; return `cache[args]` (Listing 2). Limitations: arguments must be hashable (no lists/dicts/arrays — a NumPy array argument breaks it); the cache grows without bound (`functools.lru_cache(maxsize=...)` fixes this with LRU eviction); keyword-argument order and default-value handling need care; and caching an impure function (I/O, randomness) silently changes semantics.

Q5. Explain `*args` and `**kwargs`. Why do decorators almost always use them?

In a signature, `*args` packs surplus positional arguments into a tuple and `**kwargs` packs surplus keyword arguments into a dict; at a call site the same syntax unpacks. Decorators wrap arbitrary functions, so the wrapper can't know the wrapped signature — `def wrapper(*args, **kwargs): return fn(*args, **kwargs)` forwards any call unchanged. Same idiom for subclass methods that extend a parent's (`super().__init__(**kwargs)`).

Q6. What is the GIL, and what does it mean for speeding up ML preprocessing?

The Global Interpreter Lock allows only one thread to execute Python bytecode at a time, so threading doesn't parallelize CPU-bound Python code (it does help I/O-bound work — threads waiting on network/disk release the GIL). For CPU-bound preprocessing: vectorize into NumPy/pandas (their C loops release the GIL and are the biggest win anyway), or use multiprocessing/joblib (separate processes, separate GILs — with data-serialization overhead between them). PyTorch DataLoader workers are processes for exactly this reason.

Q7. What does @property do, and where is the pattern useful in ML code?

It exposes a method as an attribute: `model.n_params` computes on access, no parentheses. Useful for derived quantities that should read like state but must stay consistent with underlying data — `scaler.n_features_` derived from `mean_.shape`, a config's computed `steps_per_epoch`. It also lets you add validation to attribute assignment later (via the setter) without changing the class's public interface.

Q8. Why does PyTorch's Dataset require `__len__` and `__getitem__`, conceptually?

Those two dunder methods make an object obey the sequence protocol: `len(ds)` and `ds[i]` work, which is all a DataLoader needs to sample indices, fetch examples (possibly in worker processes), and batch them. It's duck typing as API design: any object implementing the protocol — wrapping files, databases, arrays — plugs in with no inheritance required (subclassing Dataset is convention and type-checking sugar). `__call__` plays the same role for models and transforms: `model(x)` works because `nn.Module` defines `__call__` (which adds hooks around your `forward`).

NumPy

Q9. Why is NumPy so much faster than pure Python for numeric work? Give the memory-level answer.

A list stores pointers to boxed heap objects — each float access follows a pointer, dispatches type checks, and thrashes cache. An ndarray stores raw fixed-dtype values contiguously; a ufunc runs one compiled C loop over the buffer with sequential (prefetch-friendly, SIMD-able) access and zero per-element interpretation. Same asymptotic complexity, $\sim 100\times$ constant factor. *Interviewers listen for: boxing/pointers vs contiguous buffer, interpreter dispatch vs C loop — not just "it's in C."*

Q10. State the broadcasting rules, and give the output shape of adding (3, 1, 5) and (4, 5).

Align shapes from the trailing dimension; dimensions are compatible if equal or if either is 1 (stretched conceptually, no copy); missing leading dims count as 1. Here: (3, 1, 5) vs (1, 4, 5) $\rightarrow$ last dim $5=5$ $\checkmark$; middle 1 vs 4 $\rightarrow$ stretch to 4; leading 3 vs 1 $\rightarrow$ stretch to 3. Result: **(3, 4, 5)**. Incompatible example: (3, 2) + (3,) fails — trailing 2 vs 3.

Q11. You slice an array, modify the slice, and the original changes. Explain, and say how you'd prevent it.

Basic slicing returns a view — new shape/strides metadata over the same buffer — so writes are visible through both names (Listing 4). Prevent with an explicit `.copy()`. The asymmetric partner fact: fancy and boolean indexing return copies, so the reverse bug exists too — writing into `a[mask][0]` modifies a temporary and is lost (use `a[mask] = ...` directly, a single `_setitem_`, instead).

Q12. Compute row-wise softmax of a (n, k) score matrix in NumPy, numerically safely.

`z = S - S.max(axis=1, keepdims=True); e = np.exp(z); p = e / e.sum(axis=1, keepdims=True)`. The max subtraction prevents `exp` overflow (Chapter 1's log-sum-exp trick — softmax is shift-invariant so results are identical); `keepdims=True` keeps the reductions (n, 1) so they broadcast back across columns instead of misaligning. Both details are exactly what's being tested.

Q13. How would you find the 10 largest values in an array of 10 million, and what's the complexity?

`np.argpartition(x, -10)[-10:]` — $O(n)$ selection that guarantees the top 10 are in the last 10 positions, unordered; sort those 10 afterwards if order matters ($O(n + k \log k)$). Full `np.argsort` is $O(n \log n)$ — needless work for $k \ll n$. Pure-Python equivalent: `heapq.nlargest(10, x)`, $O(n \log k)$. Recommending partition-over-sort is the point of the question.

Q14. Your NumPy pipeline runs out of memory on an operation that "shouldn't" copy. What are the usual causes?

(1) A broadcast *output* that's huge even though inputs are small — pairwise ops like `X[:, None] - X[None, :]` materialize (n, n, d). (2) Chained expressions allocating temporaries (`(a*b + c)**2` makes several full-size intermediates; `np.multiply(a, b, out=buf)`-style or in-place ops help). (3) Silent float64 upcasts doubling footprint. (4) Fancy indexing copying where a view was assumed. Fixes: chunk the computation, restructure the algebra (norms + matmul instead of explicit differences), control dtypes, use in-place operations.

Pandas

Q15. Explain inner/left/right/outer joins. After a left join, when does the row count of the result exceed the left table's?

Inner keeps keys present in both; left keeps all left rows with NaN for unmatched right columns; right mirrors; outer is the union. A left join grows beyond `len(left)` exactly when the right table has duplicate join keys — each left row multiplies by its number of right matches (Cartesian per key). Guard with `validate='many_to_one'` (raises on unexpected duplicates) and a row-count assertion after the merge.

Q16. groupby: when do you use `transform` instead of `agg`? Give a feature-engineering example.

`agg` returns one row per group — summaries. `transform` returns a same-length result broadcast to every member row — group-relative features without a merge:

```
df['ratio_to_user_mean'] = df.amount / df.groupby('user_id').amount.transform('mean'),
```

z-scores within group, group counts as a column (`transform('size')`). If you find yourself computing `agg` and merging it back on the key, `transform` was the answer.

Q17. What is the `SettingWithCopyWarning`, and what's the correct pattern?

It fires on chained indexing assignment — `df[df.x > 0]['y'] = 1` — where the first selection may return a copy, so the write may modify a temporary and vanish. Whether it's a view or copy is an implementation detail you must not rely on. Correct: one indexer, `df.loc[df.x > 0, 'y'] = 1`. When deriving a sub-DataFrame to work on, take `.copy()` explicitly and silence the ambiguity by construction.

Q18. A column of integers gains missing values and becomes float. Why, and what are the options?

Classic pandas uses NumPy's NaN — a float — as its missing marker, and there's no NaN in fixed-width ints, so the column upcasts to float64 (ids like 10234 become 10234.0, and exact equality/joins can break). Options: nullable dtypes (`Int64`, capital I) which carry a separate validity mask; fill the missingness before casting back; or restructure so the key column never has gaps. Knowing *why* (dtype system, not a bug) is the point.

Q19. You must compute a per-row result and your `apply(axis=1)` takes minutes. Walk through the escalation ladder.

(1) Rewrite as vectorized column arithmetic — most row-wise lambdas are just column expressions in disguise (Listing 6: 2,188×). (2) Conditionals → `np.where/np.select`; membership → `.isin`; lookups → `.map(dict)`. (3) Strings/dates → `.str/.dt` accessors. (4) Drop to NumPy: `f(df.a.values, df.b.values)`. (5) If irreducibly per-row and Python, consider `itertuples()` (much faster than `apply`'s Series construction) or `parallelize/chunk`. Mention profiling first — optimize the measured bottleneck, not the assumed one.

Q20. How do you process a 100 GB CSV on a 16 GB laptop?

Stream: `pd.read_csv(..., chunksize=1_000_000)` returns an iterator of DataFrames — aggregate incrementally (running sums/counts, groupby partials merged at the end). Cut width early with `usecols`; shrink dtypes (`float32`, `category` for low-cardinality strings — often 10× on its own). Better storage: convert once to Parquet (columnar, compressed, column-pruning reads). Beyond pandas: Polars/DuckDB handle out-of-core elegantly; Spark (Chapter 28) when it's genuinely distributed. The structure of the answer — stream, shrink, change format, change engine — matters more than tool names.

Complexity and performance**Q21. Why is building a string with `+=` in a loop $O(n^2)$, and what's the fix?**

Strings are immutable: each `+=` allocates a fresh string and copies both operands, so total copying is $1+2+\dots+n \approx n^2/2$. Fix: append parts to a list (amortized $O(1)$ each) and `''.join(parts)` once — $O(n)$ total. The same immutability logic explains why repeated `np.append/pd.concat` in a loop is quadratic: collect, then concatenate once.

Q22. `x` in container — give the complexity for list, set, and sorted list, and an ML situation where the difference matters.

List: $O(n)$ scan. Set/dict: $O(1)$ average hash lookup. Sorted list with `bisect`: $O(\log n)$. Situation: filtering a million tokens against a stopwords collection — list makes it $O(n \cdot m)$, set makes it $O(n)$; or checking "have I seen this user id?" during streaming dedup. The follow-up trap: set membership requires hashable elements, so you can't put lists or NumPy arrays in a set (use tuples).

Q23. Why is `deque` the right structure for a sliding window over a stream, and what's the complexity story?

`collections.deque` is a doubly-linked block list: $O(1)$ append and pop at *both* ends, versus a list's $O(n)$ `pop(0)`. A fixed-length window is `deque(maxlen=k)` — appending auto-evicts the oldest element. Streaming feature computations (rolling means over the last k events, rate limiting, recent-history caches) are exactly this access pattern. A list-based window pays $O(k)$ per step; deque pays $O(1)$.

Q24. Estimate memory: a Python list of 10M floats vs a float64 NumPy array vs float32. Why the differences?

NumPy float64: $10\text{M} \times 8\text{ B} = 80\text{ MB}$. float32: 40 MB. Python list: $\sim 24\text{--}28\text{ B}$ per float object (object header + refcount + value) plus 8 B per list pointer $\rightarrow \sim 300\text{ MB}$, roughly $4\times$ the float64 array. This is why "load into NumPy with an appropriate dtype" is itself a memory optimization, and why float32 is the deep-learning default (Chapter 13 covers mixed precision).

Clean, testable ML code and scenarios

Q25. Your training metric changed between two runs with identical code and data. List the suspects.

Unseeded (or partially seeded) randomness: NumPy/Python/framework RNGs each need seeding, and a seed set in one process doesn't reach DataLoader workers. Nondeterministic GPU kernels (atomics; cuDNN autotune picking different algorithms). Environment drift: library versions changing defaults. Data order: filesystem listing order or hash-seed-dependent iteration feeding an order-sensitive pipeline. Uncontrolled train/val split. The systematic answer: fix all seeds, pin versions, log configs and data hashes, then diff the two runs' logs. *Interviewers listen for: multiple RNGs and worker processes, not just "set the seed."*

Q26. How would you unit-test a feature preprocessing pipeline? Name test categories with examples.

Contract/shape tests (output is (n, d) float32, column order stable, no NaNs escape); statistical property tests (standardized columns have $\text{mean} \approx 0 / \text{std} \approx 1$; encoded categories inverse-transform back); leakage tests (fit on train, transform a held-out point, verify train statistics were used — Listing 7); edge cases (constant column, empty group, unseen category at transform time, all-missing column); golden tests (tiny fixture in, pinned expected frame out); and an end-to-end smoke test on 100 rows through the whole pipeline. Small deterministic fixtures, one behavior per test.

Q27. What does "overfit one batch" test, and why is it the first debugging step for a new training loop?

Train on ~10 examples until loss approaches zero. A correctly wired model can memorize 10 points; if it can't, the bug is structural — loss connected to the wrong tensors, labels misaligned with inputs (a shuffle applied to X but not y), gradients not flowing (detached graph, wrong optimizer params), learning rate absurd, or wrong loss for the target encoding. It isolates "the code is broken" from "the model/data/hyperparameters are weak" in minutes, before any expensive full run. It's a *necessary* not sufficient check — passing it says nothing about generalization.

Q28. A teammate's 400-line training script works but nobody can modify it safely. What refactor do you propose, concretely?

Extract the stage boundaries first:

`load_data()` → `build_features()` → `build_model(cfg)` → `train(model, data, cfg)` → `evaluate(model, data)`, each pure-ish with explicit inputs/outputs; hyperparameters and paths into one dataclass config (constructible from YAML/CLI); randomness passed in as seeded generators; I/O quarantined at the edges. Then lock behavior with a golden test (fixed seed, tiny data, pinned metric) *before* refactoring, refactor incrementally, and keep the test green. The estimator pattern and the golden-test-first move are what the question screens for.

Chapter 3: Data Structures & Algorithms (ML-flavored)

The coding round in an ML interview loop is usually a standard software-engineering screen wearing an ML costume. You will be asked to manipulate arrays and strings, count things with hash maps, keep a running top-k with a heap, traverse a graph, or write a short dynamic program — and then, in the ML-specific variant, to implement KNN, k-means, or a sampling routine from scratch with no library beyond NumPy. Interviewers use this round to answer one question: *when this person's model pipeline breaks at 2 a.m., can they reason about code, or only call `fit()`?*

This chapter covers the data structures and algorithms that actually appear in ML coding screens, in the order you should learn them: linear structures (arrays, strings, hash maps, stacks, queues, heaps), hierarchical and relational structures (trees, graphs, BFS/DFS), the two algorithmic workhorses (sorting and searching), dynamic programming at the level interviews test it, and finally the ML-specific implementations — KNN, k-means, and sampling algorithms — that separate ML candidates from generic software candidates. Chapter 2 covered *what Python's containers cost*; this chapter covers *what to build with them*. Throughout, every pattern is tied to where it shows up in real ML work, because that is exactly the framing a strong candidate uses out loud.

Arrays and strings

An array is a contiguous block of memory indexed in $O(1)$; in Python interviews the list plays this role, and in ML code the NumPy ndarray is its typed, vectorized descendant. Strings are immutable character arrays — every "modification" allocates a new string, which is why building a string in a loop with `+=` is $O(n^2)$ and `''.join(parts)` is $O(n)$. Most array and string questions are not about the structure itself but about avoiding the naive $O(n^2)$ double loop with one of three patterns.

Two pointers. Keep two indices that move through the array under some invariant. Canonical forms: pointers converging from both ends (pair-with-target-sum in a sorted array, reversing in place, container-with-most-water), and slow/fast pointers moving in the same direction (in-place deduplication of a sorted array, partitioning). The invariant is what you should narrate in an interview: "everything left of `slow` is the deduplicated prefix." Two-pointer solutions turn $O(n^2)$ pair-scans into $O(n)$ whenever sortedness or monotonicity lets you rule out candidates wholesale.

Sliding window. A two-pointer special case for contiguous subarrays and substrings: expand the right edge, and contract the left edge only when a constraint breaks (window sum exceeds a budget, a character repeats). Each element enters and leaves the window

at most once — $O(n)$ total. This is the pattern behind "longest substring without repeating characters," "smallest subarray with sum $\geq s$," and, in ML terms, any rolling-statistic computation: a moving average over a metric stream is a fixed-size sliding window, and rolling means/medians over time series features are its production form.

Prefix sums. Precompute `prefix[i]` = sum of the first i elements; any range sum becomes `prefix[j] - prefix[i]` in $O(1)$ after $O(n)$ setup. The trade — linear preprocessing for constant-time queries — generalizes far beyond sums (prefix max, prefix counts) and is the 1-D ancestor of integral images in computer vision, which answer any rectangular patch-sum query in $O(1)$ exactly this way.

For strings specifically, interviewers add frequency-count questions (anagrams, palindrome permutations) that are really hash-map questions, and parsing questions (tokenize, validate) that are really stack questions. Genuinely string-specific algorithms (KMP, suffix arrays) are rare in ML loops; frequency counting plus two pointers covers the large majority. The ML echo is everywhere: tokenization is string splitting with rules, vocabulary construction is frequency counting, and n -gram extraction is a sliding window over tokens.

Hash maps and sets

Chapter 2 covered how a hash table works (hash $\rightarrow$ slot $\rightarrow$ probe on collision; $O(1)$ average, $O(n)$ adversarial worst case). This section is about the *patterns* — a hash map is the answer to more interview questions than any other structure, because it converts "have I seen this before?" and "how many of these are there?" from $O(n)$ scans into $O(1)$ lookups.

Seen-before. Two-sum is the archetype: for each x , ask whether `target - x` was already seen. One pass, $O(n)$ time, $O(n)$ space — versus $O(n^2)$ for the double loop or $O(n \log n)$ for sort-then-two-pointers. The general move: when a double loop's inner scan is *searching for a value*, replace the scan with a dict.

Counting. `collections.Counter` builds a frequency table in one $O(n)$ pass; anagram checks compare two Counters; "group anagrams" keys a dict by the sorted word (or a 26-tuple of counts). In ML this pattern is load-bearing: building a vocabulary is `Counter(tokens).most_common(vocab_size)`, class-imbalance diagnosis is `Counter(labels)`, and TF (term frequency) in TF-IDF is literally a per-document Counter.

Grouping / indexing. A dict of lists (`defaultdict(list)`) groups records by key in $O(n)$ — the in-memory version of SQL's GROUP BY and the first half of a hash join. An inverted index (token $\rightarrow$ list of document ids) is the same pattern and is the core data structure of classical retrieval; it reappears in Chapter 24 as the sparse half of hybrid RAG search.

Two costs to volunteer before the interviewer asks: hash maps buy speed with memory ($O(n)$ extra) and lose ordering (you cannot ask a dict for "the smallest key" without scanning — that is what heaps and sorted structures are for). And keys must be hashable:

a NumPy array cannot key a dict; convert to `tuple(arr)` or `arr.tobytes()` first — a detail that bites in memoized feature pipelines.

Stacks and queues

A **stack** is last-in-first-out: `append` and `pop` on the end of a Python list, both $O(1)$. It is the structure of *nesting* — matched parentheses, undo histories, recursion. Every recursive algorithm implicitly uses the call stack, and converting recursion to an explicit stack is a standard interview follow-up ("now do it iteratively") as well as the practical fix for `RecursionError` on deep structures — a depth-10,000 decision tree will blow Python's default ~1,000-frame recursion limit during traversal, and the cure is an explicit stack. The framework echo is exact: autodiff engines record operations on a tape and pop them in reverse during backpropagation — the backward pass is a stack traversal of the forward computation.

The classic hard-stack question is the **monotonic stack**: maintain a stack whose elements are kept sorted by popping everything that violates the order before each push ("next greater element," "largest rectangle in histogram"). Each element is pushed and popped at most once — $O(n)$ for what looks like an $O(n^2)$ problem.

A **queue** is first-in-first-out: use `collections.deque` (`append` / `popleft`, both $O(1)$); `list.pop(0)` is $O(n)$ and naming that cost is a free point in interviews. Queues are the structure of *fair processing in arrival order*: BFS frontiers (next section), producer-consumer buffers. That producer-consumer shape is exactly a training input pipeline — `DataLoader` workers put prepared batches on a queue, the training loop gets them; the queue decouples the two speeds. A bounded deque (`deque(maxlen=k)`) is the natural container for "last k checkpoints" or a rolling window of recent losses.

Heaps and priority queues

A heap answers a question neither a dict nor a sorted list answers well: *repeatedly give me the smallest (or largest) item as items keep arriving*. A binary min-heap is a complete binary tree, stored flat in an array (children of index i at $2i+1$ and $2i+2$ — no pointers), with one invariant: every parent $\leq$ its children. The root is the minimum, readable in $O(1)$; `push` and `pop` restore the invariant by bubbling an element up or down one level at a time, $O(\log n)$ each; building a heap from n items in one shot (`heapify`) is $O(n)$, cheaper than n pushes.

Python's `heapq` is a min-heap over lists. Two idioms to know cold: for a max-heap, push negated values (or negate a numeric key); for compound items, push tuples (`key`, `payload`) — `heapq` compares tuples lexicographically, so ties on the key fall through to comparing payloads, which crashes if payloads aren't comparable; insert a tiebreaker counter (`key`, `i`, `payload`).

Top-k, the signature ML pattern. To keep the k largest of a stream: maintain a *min*-heap of size k; each new item is compared against the heap's root (the smallest of the current top-k) and replaces it if larger — `heapq.heappushpop` does this in one $O(\log k)$ step. Total $O(n \log k)$ time, $O(k)$ memory, versus $O(n \log n)$ and $O(n)$ for full sorting. The inversion — a min-heap to track *largest* items — is the part candidates fumble; say out loud that the root is "the weakest member of the club, first to be evicted." This pattern is everywhere in ML: k nearest neighbors from a distance stream, beam search (keep the k best partial sequences per step — Chapter 22), top-k tokens before sampling, best-k checkpoints by validation loss. `heapq.nlargest(k, xs)` packages it.

Merge k sorted streams. A heap of size k holding the current head of each stream pops the global minimum in $O(\log k)$ — the interview form is "merge k sorted lists," the systems form is external-merge sort over shards too big for memory (Chapter 28's territory). Dijkstra's shortest-path algorithm is the same idea: a priority queue always expands the closest unexplored node.

Trees

A tree is a connected graph with no cycles: n nodes, n−1 edges, one root, every non-root node with exactly one parent. Interview trees are usually **binary trees** (≤ 2 children), and the vocabulary is fair game: *depth* of a node (edges from root), *height* of the tree, *leaf*, *balanced* (heights of any node's subtrees differ by at most 1). A **binary search tree (BST)** adds the ordering invariant — everything in the left subtree $<$ node $<$ everything in the right subtree — which makes search, insert, and delete $O(h)$, where h is the height: $O(\log n)$ if balanced, degrading to $O(n)$ if the tree becomes a linked list (insert sorted data into a naive BST and it does exactly that; production structures like red-black trees rebalance to keep h logarithmic, and knowing *that they exist and why* is the expected depth).

Traversals are the heart of tree questions, and nearly all are four lines of recursion. Depth-first comes in three orders, named for where the node is visited relative to its children: **preorder** (node, left, right — serializing/copying a tree), **inorder** (left, node, right — which on a BST yields sorted order, the single most-tested tree fact), and **postorder** (left, right, node — children before parents: deleting a tree, computing subtree aggregates). **Level-order** is breadth-first with a queue. Most "hard" tree questions are one traversal plus bookkeeping: validate a BST (inorder must be increasing, or pass down min/max bounds), max depth (postorder), lowest common ancestor.

Trees are load-bearing in ML. A decision tree *is* a binary tree — prediction is root-to-leaf descent by feature thresholds, $O(\text{depth})$; training (Chapter 6) is recursive splitting, a build-the-tree recursion; and feature importances are computed by a postorder-style aggregation over split gains. Gradient-boosted forests are collections of hundreds of such trees. KD-trees and ball trees partition feature space to accelerate nearest-neighbor

search from $O(n)$ toward $O(\log n)$ per query in low dimensions — sklearn's `KNeighborsClassifier` chooses among brute force, KD-tree, and ball tree — though in high dimensions partitioning stops pruning anything (the curse of dimensionality, Chapter 4) and brute force or approximate methods (Chapter 24's HNSW) win. Parse trees and abstract syntax trees are how NLP represented structure before transformers flattened everything into attention.

Graphs, BFS and DFS

A graph is nodes plus edges — directed or undirected, weighted or not, possibly cyclic. Interviews expect two representations and their trade-off: the **adjacency list** (dict mapping node $\rightarrow$ list of neighbors; $O(V+E)$ space; the default) and the **adjacency matrix** ($V \times V$ array; $O(V^2)$ space; $O(1)$ edge lookup; only sensible for dense or small graphs). Real graphs are usually sparse, so adjacency lists win. Two special cases matter: a tree is a connected acyclic graph, and a **DAG** (directed acyclic graph) is the shape of every dependency system — including every neural network's computation graph and every feature pipeline.

BFS explores level by level with a queue: pop a node, push unvisited neighbors, repeat. Because it expands outward one edge-layer at a time, BFS finds *shortest paths in unweighted graphs* — that is its superpower and the reason to choose it. **DFS** dives deep with recursion or an explicit stack, and is the natural fit for exhaustive exploration: connected components, cycle detection, backtracking, topological sort. Both are $O(V+E)$ time; both require a `visited` set, and forgetting it is the classic infinite-loop bug on any graph with a cycle (on trees you can skip it only because trees have none). Choose by question: "shortest/fewest steps" $\rightarrow$ BFS; "all paths / does a path exist / components / ordering" $\rightarrow$ DFS. Memory differs too — BFS holds a frontier (up to $O(V)$ wide), DFS holds a path (up to $O(V)$ deep, hence recursion limits on long chains).

Topological sort linearizes a DAG so every edge points forward: either DFS-with-postorder-reversal, or Kahn's algorithm (repeatedly remove zero-in-degree nodes using a queue; if nodes remain un-removed, the graph has a cycle — which is how you *detect* cycles in dependency systems). This is the algorithm ML infrastructure runs on: a feature pipeline's "compute features before the model that consumes them" is a topological order; Airflow schedules DAG tasks this way (Chapter 28); and autodiff frameworks execute the forward pass in topological order of the computation graph, then backpropagate in exactly the reverse order (Chapter 12). "Backprop visits nodes in reverse topological order" is a sentence that upgrades a coding answer into an ML answer.

Weighted shortest paths (Dijkstra — BFS with a priority queue instead of a queue) and connectivity under merging (union-find) are worth recognizing by name, but ML loops rarely go deeper than BFS/DFS/topo-sort plus their applications: label propagation over a

similarity graph is BFS flood-fill, and graph neural networks (Chapter 25's neighborhood) aggregate neighbors — one BFS layer per GNN layer.

Sorting and searching

You will almost never implement a sort in production — `sorted()` exists — but interviews test sorting because it is the cleanest window into algorithmic reasoning, and one implementation question ("implement quicksort/mergesort") is common as a warm-up.

The comparison-sort landscape in one pass: **mergesort** splits in half, sorts each, and merges — $O(n \log n)$ *guaranteed*, stable, but $O(n)$ extra memory; it is the shape of external sorting when data exceeds RAM. **Quicksort** picks a pivot, partitions smaller/larger, and recurses — $O(n \log n)$ average with excellent constants and in-place partitioning, but $O(n^2)$ worst case on adversarial pivots (mitigated by random pivot choice). **Heapsort** is $O(n \log n)$ guaranteed and in-place but slower in practice and unstable. Python's built-in **Timsort** is a mergesort-insertion hybrid exploiting existing runs: $O(n \log n)$ worst case, $O(n)$ on nearly-sorted data, stable. **Stability** — equal keys keep their original order — is the practically important property: it is what makes multi-key sorting composable (sort by secondary key, then stable-sort by primary) and preserves temporal order when you sort event logs by user, which matters for leakage-free time splits (Chapter 4). Counting-based sorts (counting/radix/bucket) beat $O(n \log n)$ by not comparing at all, and $O(n \log n)$ is a proven lower bound for comparison sorts — a classic follow-up.

Two sorting-adjacent tools earn their keep in ML code. **argsort** — sort the *indices*, not the values — is how you rank one array by another: `np.argsort(scores)[-1:-1]` orders candidates by score, `np.argsort` on distances gives nearest neighbors, and feature importances are displayed via `argsort`. **Quickselect** finds the k -th smallest element in $O(n)$ average by running quicksort's partition but recursing into only one side — `np.partition/np.argpartition` expose it, and it is the right way to get top- k when k is large or to compute a median (robust statistics, Chapter 9's outlier handling) without a full sort.

Binary search is the highest-value algorithm per line of code in this chapter: on any sorted (more generally, *monotonic*) structure, halve the search space per step, $O(\log n)$. The implementation is famously bug-prone — the interview-safe invariant form keeps `[lo, hi)` as "the range that could still contain the answer," computes `mid = (lo + hi) // 2`, and moves one boundary per step; off-by-one errors and infinite loops come from mixing inclusive and exclusive conventions mid-function. `bisect.bisect_left/right` are the library forms and also the tool for "insert into sorted list" and "count items $\leq x$ " (which is how you binary-search a CDF — see weighted sampling below). The pattern's real power is **binary search on the answer**: if a yes/no feasibility check is monotonic in some parameter, binary-search the parameter itself — "smallest capacity that ships packages in D days" in interview form; threshold tuning against a monotonic constraint ("largest

classification threshold with $\text{recall} \geq 0.9$ " — precision-recall tradeoffs, Chapter 10) and learning-rate range probes in ML form. Say the magic word *monotonic* — it is the property that licenses the halving.

Dynamic programming basics

Dynamic programming applies when a problem has two properties: **optimal substructure** (the best solution is composed of best solutions to subproblems) and **overlapping subproblems** (naive recursion solves the same subproblem exponentially many times). DP is recursion plus a cache — nothing more mystical than that. Fibonacci is the toy case: naive recursion is $O(2^n)$ because $\text{fib}(k)$ is recomputed everywhere; caching results makes it $O(n)$.

Two mechanical styles. **Top-down (memoization)**: write the natural recursion, add `@functools.lru_cache` (Chapter 2's decorator, now load-bearing). **Bottom-up (tabulation)**: fill a table from base cases upward with a loop, which avoids recursion limits and often allows compressing the table to its last row or two — $O(n)$ space becoming $O(1)$ for Fibonacci-shaped recurrences. Interviewers commonly accept either and then ask for the space optimization.

The recipe that solves most interview DP: (1) define the state in words — " $\text{dp}[i]$ = best value achievable using the first i items"; (2) write the recurrence — how $\text{dp}[i]$ follows from smaller states; (3) identify base cases; (4) fix the fill order so dependencies are ready; (5) read off the answer. The classic families, each worth one worked pass: 1-D sequence DP (climbing stairs, house robber — dp over a prefix), **longest common subsequence / edit distance** (2-D table over two sequence prefixes), and **0/1 knapsack** (choose or skip each item under a budget — the shape of feature selection under a cost constraint, though in practice we use greedier methods).

Edit distance is the one to know deeply, because it is genuinely used: $\text{dist}(i, j)$ = cost of turning the first i characters of one string into the first j of another; if the current characters match, inherit $\text{dist}(i-1, j-1)$, else $1 + \min(\text{insert}, \text{delete}, \text{substitute})$. It powers spelling correction, fuzzy string matching in data cleaning and entity resolution, and — in bioinformatics dress — sequence alignment. Its deeper ML relatives share the exact same table-filling structure: dynamic time warping aligns two time series (Chapter 31), the Viterbi algorithm finds the best hidden-state path in an HMM (Chapter 19), and CTC loss for speech recognition marginalizes over alignments — all are "fill a 2-D table by a local recurrence." Beam search (Chapter 22) is what you do when the exact DP is intractable: keep only the k best states per column. And the Bellman equation of reinforcement learning (Chapter 32) is a DP recurrence over states — the term "dynamic programming" comes from Bellman himself.

ML-specific coding: KNN, k-means, and sampling from scratch

The ML round's favorite question type: "implement X from scratch, NumPy only." It tests whether you understand the algorithm well enough to state it as array operations, and whether Chapter 2's vectorization reflexes are real. Three families dominate; the full implementations, executed with output, are in the listings below.

KNN (Listing 9) is pure computation, no training: predict by majority vote (or average, for regression) of the k nearest training points. The from-scratch skeleton is: pairwise distances (vectorized via broadcasting or the expansion of the squared-distance formula), `argpartition` for the k smallest per query — an $O(n)$ selection rather than an $O(n \log n)$ sort, exactly quickselect earning its keep — then a vote. Complexity is the punchline interviewers want: $O(n \cdot d)$ *per query* at prediction time and $O(1)$ training, the inverse of parametric models; the space-partitioning escape hatches (KD-trees, and their high-dimensional failure) connect back to the trees section.

K-means (Listing 10) is the two-step dance: assign every point to its nearest centroid, recompute each centroid as the mean of its assigned points, repeat until assignments stop changing. Each step is one broadcasting expression; the pedagogical payload is that both steps *provably* never increase the within-cluster sum of squares, so the loop converges — to a local optimum that depends on initialization (hence k-means++ and multiple restarts, Chapter 8). Edge case to handle out loud: a centroid that loses all its points (reseed it randomly).

Sampling algorithms are the sleeper topic — they look niche but appear constantly, because training *is* sampling. Four are interview staples. *Fisher-Yates shuffle* (Listing 11): swap position i with a uniformly random position in $[i, n)$ — $O(n)$, exactly uniform over permutations; it is what every `shuffle=True` runs, and the naive "sort by random key" alternative is $O(n \log n)$ while "swap with a random position in $[0, n)$ " is subtly *non-uniform*, a known trap. *Reservoir sampling* (Listing 11): keep a uniform sample of k items from a stream of unknown length — keep the first k , then accept item i with probability k/i , evicting a random resident; the proof by induction that every item ends up with probability k/n is a classic whiteboard follow-up; production form: sampling from a data stream too large to hold. *Weighted sampling* (Listing 12): build the cumulative distribution and binary-search a uniform random number into it — $O(\log n)$ per draw after $O(n)$ setup; this is temperature sampling from a softmax (Chapter 22), boosting's example reweighting (Chapter 7), and prioritized replay (Chapter 32). *Stratified splitting* (Listing 12): group indices by label (a hash-map grouping), sample within each group — the correct train/test split under class imbalance (Chapter 4).

Code implementations

All listings below were executed as shown; the output blocks are real. Run them, break them, and re-derive them — in a live screen you will be writing these from memory.

Listing 1 — Two pointers and sliding window

Four $O(n)$ patterns that replace $O(n^2)$ double loops. Narrate the invariant: everything left of `slow` is finished work; the window `[left, right]` always satisfies (or has just broken) the constraint.

```

"""Listing 1 -- Two pointers and sliding window."""

def dedup_sorted(a):
    """In-place dedup of a sorted list. Invariant: a[:slow] is the deduplicated prefix."""
    if not a:
        return 0
    slow = 1
    for fast in range(1, len(a)):
        if a[fast] != a[slow - 1]:
            a[slow] = a[fast]
            slow += 1
    return slow
    # next write position
    # fast scans every element
    # new value -> keep it
    # length of deduplicated prefix

def pair_with_sum(a, target):
    """Sorted array: find a pair summing to target. Converging pointers, O(n)."""
    lo, hi = 0, len(a) - 1
    while lo < hi:
        s = a[lo] + a[hi]
        if s == target:
            return a[lo], a[hi]
        if s < target:
            lo += 1
            # sum too small -> only moving lo up can help
        else:
            hi -= 1
            # sum too big -> only moving hi down can help
    return None

def smallest_window_geq(a, s):
    """Length of the smallest contiguous window with sum >= s. O(n)."""
    best = float('inf')
    left = wsum = 0
    for right, x in enumerate(a):
        wsum += x
        while wsum >= s:
            best = min(best, right - left + 1)
            wsum -= a[left]
            left += 1
    return best if best != float('inf') else 0

def rolling_mean(xs, k):
    """Moving average via a running window sum -- O(n), not O(n*k)."""
    out, wsum = [], 0.0
    for i, x in enumerate(xs):
        wsum += x
        if i >= k:
            wsum -= xs[i - k]
            # element leaves the window exactly once
        if i >= k - 1:
            out.append(wsum / k)
    return out

a = [1, 1, 2, 3, 3, 3, 5]

```



```
n = dedup_sorted(a)
print("dedup:", a[:n])
print("pair summing to 8 in [1,2,4,6,7]:", pair_with_sum([1, 2, 4, 6, 7], 8))
print("smallest window >= 7 in [2,3,1,2,4,3]:", smallest_window_geq([2, 3, 1, 2, 4, 3], 7))
print("rolling mean k=3 of [1..5]:", rolling_mean([1, 2, 3, 4, 5], 3))
```

Output:

```
dedup: [1, 2, 3, 5]
pair summing to 8 in [1,2,4,6,7]: (1, 7)
smallest window >= 7 in [2,3,1,2,4,3]: 2
rolling mean k=3 of [1..5]: [2.0, 3.0, 4.0]
```

Listing 2 — Hash-map patterns: seen-before, counting, grouping

The three moves that answer most hash-map questions, ending with the ML form: vocabulary construction is the counting pattern verbatim.

```
"""Listing 2 -- Hash-map patterns: seen-before, counting, grouping."""
from collections import Counter, defaultdict

def two_sum(nums, target):
    """Indices of two numbers summing to target. One pass, O(n) time/space."""
    seen = {} # value -> index
    for i, x in enumerate(nums):
        if target - x in seen: # the O(1) lookup that replaces the inner loop
            return seen[target - x], i
        seen[x] = i
    return None

def group_anagrams(words):
    """Key each word by its sorted letters; anagrams collide on the same key."""
    groups = defaultdict(list)
    for w in words:
        groups[''.join(sorted(w))].append(w)
    return list(groups.values())

def build_vocab(tokens, size):
    """Counting pattern: top-`size` tokens by frequency -> token->id map."""
    counts = Counter(tokens) # one O(n) pass
    most = counts.most_common(size) # heap-based top-k inside
    return {tok: i for i, (tok, _) in enumerate(most)}

print("two_sum([2,7,11,15], 9):", two_sum([2, 7, 11, 15], 9))
print("anagram groups:", group_anagrams(['eat', 'tea', 'tan', 'ate', 'nat', 'bat']))
toks = "the cat sat on the mat the cat".split()
print("vocab (size 3):", build_vocab(toks, 3))
```

Output:

```
two_sum([2,7,11,15], 9): (0, 1)
anagram groups: [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
vocab (size 3): {'the': 0, 'cat': 1, 'sat': 2}
```

Listing 3 — Heaps: streaming top-k and merge-k

The signature inversion — a *min*-heap tracks the *largest* k items, its root the weakest member and first to be evicted. Tuples `(value, list_id, index)` give `heapq` a total order with no custom comparator.

```
"""Listing 3 -- Heaps: streaming top-k and merging k sorted streams."""
import heapq

def top_k_stream(stream, k):
    """Keep the k largest items of a stream. Min-heap of size k, O(n log k)."""
    heap = []
    for x in stream:
        if len(heap) < k:
            heapq.heappush(heap, x)
        elif x > heap[0]:
            # beats the weakest current member?
            # evict root, insert x -- one O(log k) op
            heapq.heappushpop(heap, x)
    return sorted(heap, reverse=True)

def merge_k_sorted(lists):
    """Merge k sorted lists. Heap holds one head per list; pop = global min."""
    heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst] # (value, list_id,
index)
    heapq.heapify(heap) # O(k)
    out = []
    while heap:
        val, i, j = heapq.heappop(heap) # O(log k)
        out.append(val)
        if j + 1 < len(lists[i]):
            heapq.heappush(heap, (lists[i][j + 1], i, j + 1))
    return out

scores = [0.11, 0.93, 0.35, 0.88, 0.62, 0.99, 0.41, 0.77]
print("top-3 scores:", top_k_stream(scores, 3))
print("merge:", merge_k_sorted([[1, 4, 9], [2, 3, 11], [5, 6, 7]]))
```

Output:

```
top-3 scores: [0.99, 0.93, 0.88]
merge: [1, 2, 3, 4, 5, 6, 7, 9, 11]
```

Listing 4 — Tree traversals and BST validation

Each traversal is four lines; the payload is knowing which order fits which job. The corruption test shows why 'check each node against its children' is the classic wrong answer — validity is a *subtree-wide* bounds condition.

```
"""Listing 4 -- Tree traversals and BST validation."""
from collections import deque

class Node:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def inorder(node, out):
    """left, node, right -- sorted order on a BST."""
    if node:
        inorder(node.left, out)
```

```

        out.append(node.val)
        inorder(node.right, out)

def max_depth(node):
    """Postorder aggregation: children first, then combine."""
    if node is None:
        return 0
    return 1 + max(max_depth(node.left), max_depth(node.right))

def level_order(root):
    """BFS with a queue; one output list per level."""
    if not root:
        return []
    levels, q = [], deque([root])
    while q:
        levels.append([n.val for n in q])
        q = deque(c for n in q for c in (n.left, n.right) if c)
    return levels

def is_bst(node, lo=float('-inf'), hi=float('inf')):
    """Pass down the (lo, hi) bounds each node must satisfy."""
    if node is None:
        return True
    if not (lo < node.val < hi):
        return False
    return is_bst(node.left, lo, node.val) and is_bst(node.right, node.val, hi)

#
#      8
#     / \
#    3   10
#   / \  \
#  1  6   14
root = Node(8, Node(3, Node(1), Node(6)), Node(10, None, Node(14)))
out = []
inorder(root, out)
print("inorder:", out)           # sorted ==> valid BST
print("max depth:", max_depth(root))
print("levels:", level_order(root))
print("is_bst:", is_bst(root))
root.left.right.val = 9          # 9 > 8 but sits in the left subtree
print("is_bst after corruption:", is_bst(root))

```

Output:

```

inorder: [1, 3, 6, 8, 10, 14]
max depth: 3
levels: [[8], [3, 10], [1, 6, 14]]
is_bst: True
is_bst after corruption: False

```

Listing 5 — Graphs: BFS shortest path, DFS components, topological sort

The visited set does double duty as the BFS distance map. DFS is iterative with an explicit stack — no recursion limit. Kahn's algorithm both orders the feature-pipeline DAG and detects the cycle when a back-edge is added.

```

"""Listing 5 -- BFS shortest path, DFS components, Kahn's topological sort."""
from collections import deque, defaultdict

def bfs_shortest(adj, src, dst):

```

```

"""Fewest-edges path length in an unweighted graph."""
q, dist = deque([src]), {src: 0}
while q:
    u = q.popleft()
    if u == dst:
        return dist[u]
    for v in adj[u]:
        if v not in dist:
            dist[v] = dist[u] + 1
            q.append(v)
return -1

def components(adj, nodes):
    """Connected components via iterative DFS (explicit stack, no recursion limit)."""
    seen, comps = set(), []
    for s in nodes:
        if s in seen:
            continue
        stack, comp = [s], []
        seen.add(s)
        while stack:
            u = stack.pop()
            comp.append(u)
            for v in adj[u]:
                if v not in seen:
                    seen.add(v)
                    stack.append(v)
        comps.append(sorted(comp))
    return comps

def topo_sort(edges):
    """Kahn's algorithm. Returns order, or raises if a cycle exists."""
    adj, indeg = defaultdict(list), defaultdict(int)
    nodes = set()
    for u, v in edges:
        adj[u].append(v)
        indeg[v] += 1
        nodes |= {u, v}
    q = deque(sorted(n for n in nodes if indeg[n] == 0))
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    if len(order) != len(nodes):
        raise ValueError("cycle detected -- not a DAG")
    return order

adj = {1: [2, 3], 2: [1, 4], 3: [1], 4: [2], 5: [6], 6: [5]}
print("shortest 3->4:", bfs_shortest(adj, 3, 4))
print("components:", components(adj, sorted(adj)))

# A feature pipeline as a DAG: raw -> cleaned -> {features, labels} -> model
pipeline = [("raw", "cleaned"), ("cleaned", "features"),
            ("cleaned", "labels"), ("features", "model"), ("labels", "model")]
print("pipeline order:", topo_sort(pipeline))
try:
    topo_sort(pipeline + [("model", "raw")])
except ValueError as e:
    print("with back-edge:", e)

```

Output:

```

shortest 3->4: 3
components: [[1, 2, 3, 4], [5, 6]]
pipeline order: ['raw', 'cleaned', 'features', 'labels', 'model']
with back-edge: cycle detected -- not a DAG

```

Listing 6 — Binary search: the invariant form and search on the answer

The half-open `[lo, hi)` convention removes the off-by-one bugs; `smallest_capacity` shows the pattern's real power — binary-searching a *parameter* whose feasibility check is monotonic.

```

"""Listing 6 -- Binary search: the invariant form, and search on the answer."""
import bisect

def binary_search(a, target):
    """Half-open invariant: the answer, if present, is always in [lo, hi)."""
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] == target:
            return mid
        if a[mid] < target:
            lo = mid + 1           # mid ruled out -> exclude it
        else:
            hi = mid              # a[mid] too big; hi stays exclusive
    return -1

def smallest_capacity(weights, days):
    """Search on the answer: min capacity to ship `weights` in `days` days.
    feasible(c) is monotonic in c -- that is what licenses the halving."""
    def feasible(cap):
        d, load = 1, 0
        for w in weights:
            if load + w > cap:
                d, load = d + 1, 0
            load += w
        return d <= days
    lo, hi = max(weights), sum(weights)   # answer bracketed in [lo, hi]
    while lo < hi:
        mid = (lo + hi) // 2
        if feasible(mid):
            hi = mid                  # mid works -> try smaller
        else:
            lo = mid + 1             # mid fails -> need bigger
    return lo

a = [2, 5, 8, 12, 16, 23, 38]
print("index of 16:", binary_search(a, 16))
print("index of 17:", binary_search(a, 17))
print("insertion point for 17:", bisect.bisect_left(a, 17))
print("min capacity, [1..10] in 5 days:", smallest_capacity(list(range(1, 11)), 5))

```

Output:

```

index of 16: 4
index of 17: -1
insertion point for 17: 5
min capacity, [1..10] in 5 days: 15

```

Listing 7 — Mergesort and quickselect

Mergesort as the canonical guaranteed- $O(n \log n)$ stable sort (the `<=` in the merge is where stability lives); quickselect finds the median in $O(n)$ average by recursing into only one partition.

```

"""Listing 7 -- Mergesort and quickselect."""
import random

def mergesort(a):
    """O(n log n) guaranteed, stable, O(n) extra memory."""
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left, right = mergesort(a[:mid]), mergesort(a[mid:])
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]: # `<=` is what makes the sort stable
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    return out + left[i:] + right[j:]

def quickselect(a, k):
    """k-th smallest (0-indexed), O(n) average: partition, recurse one side only."""
    if len(a) == 1:
        return a[0]
    pivot = random.choice(a) # random pivot defends against O(n^2)
    lt = [x for x in a if x < pivot]
    eq = [x for x in a if x == pivot]
    gt = [x for x in a if x > pivot]
    if k < len(lt):
        return quickselect(lt, k)
    if k < len(lt) + len(eq):
        return pivot
    return quickselect(gt, k - len(lt) - len(eq))

random.seed(0)
xs = [7, 2, 9, 4, 4, 1, 8, 3]
print("mergesort:", mergesort(xs))
print("median via quickselect:", quickselect(xs, len(xs) // 2))

```

Output:

```

mergesort: [1, 2, 3, 4, 4, 7, 8, 9]
median via quickselect: 4

```

Listing 8 — Dynamic programming: memoized Fibonacci and edit distance

Top-down DP is one decorator; bottom-up edit distance is the 2-D table every alignment algorithm (DTW, Viterbi, CTC) generalizes. `fib(80)` would take $\sim 2^{80}$ calls naively; memoized it takes 81.

```

"""Listing 8 -- Dynamic programming: memoized Fibonacci and edit distance."""
from functools import lru_cache

```

```

@lru_cache(maxsize=None)          # recursion + cache = top-down DP
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

def edit_distance(s, t):
    """dp[i][j] = min ops to turn s[:i] into t[:j]. Bottom-up, O(len(s)*len(t))."""
    m, n = len(s), len(t)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i                # delete all of s[:i]
    for j in range(n + 1):
        dp[0][j] = j                # insert all of t[:j]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s[i - 1] == t[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]          # match: free
            else:
                dp[i][j] = 1 + min(dp[i - 1][j],      # delete from s
                                   dp[i][j - 1],      # insert into s
                                   dp[i - 1][j - 1])   # substitute

    return dp[m][n]

print("fib(80):", fib(80))          # naive recursion: ~2^80 calls; memoized: 81
print("edit('kitten','sitting'):", edit_distance("kitten", "sitting"))
print("edit('gradient','gradinet'):", edit_distance("gradient", "gradinet"))

```

Output:

```

fib(80): 23416728348467685
edit('kitten','sitting'): 3
edit('gradient','gradinet'): 2

```

Listing 9 — KNN from scratch

No training loop, no Python loops over points: pairwise distances via the squared-norm expansion, k-smallest via `argpartition` (O(n) selection), then a vote. 96% held-out accuracy on two Gaussian blobs confirms it works.

```

"""Listing 9 -- KNN from scratch, NumPy only."""
import numpy as np

def knn_predict(X_train, y_train, X_query, k):
    """Vectorized k-nearest-neighbors classification.
    Distances via the expansion ||a-b||^2 = ||a||^2 + ||b||^2 - 2 a.b -- no loops."""
    # (q, n) matrix of squared distances between every query and training point
    d2 = (np.sum(X_query**2, axis=1)[:, None]          # (q, 1)
          + np.sum(X_train**2, axis=1)[None, :]       # (1, n) -> broadcasts to (q, n)
          - 2 * X_query @ X_train.T)                  # (q, n)
    # k smallest per row: argpartition is O(n) selection, not an O(n log n) sort
    nn = np.argpartition(d2, k, axis=1)[:k]           # (q, k) neighbor indices
    votes = y_train[nn]                               # (q, k) neighbor labels
    # majority vote per query row
    return np.array([np.bincount(row).argmax() for row in votes])

rng = np.random.default_rng(42)
# two Gaussian blobs: class 0 near (0,0), class 1 near (3,3)
X0 = rng.normal(0, 1, (50, 2)); X1 = rng.normal(3, 1, (50, 2))
X = np.vstack([X0, X1]); y = np.array([0]*50 + [1]*50)
Xq = np.array([[0.0, 0.0], [3.0, 3.0], [1.5, 1.5]])
print("predictions:", knn_predict(X, y, Xq, k=5))

```

```
# sanity: accuracy on held-out points from the same blobs
Xt0 = rng.normal(0, 1, (100, 2)); Xt1 = rng.normal(3, 1, (100, 2))
Xt = np.vstack([Xt0, Xt1]); yt = np.array([0]*100 + [1]*100)
acc = (knn_predict(X, y, Xt, k=5) == yt).mean()
print(f"held-out accuracy: {acc:.3f}")
```

Output:

```
predictions: [0 1 1]
held-out accuracy: 0.960
```

Listing 10 — k-means from scratch

Assign and update are each one broadcasting expression. Both steps provably never increase inertia, so the loop converges — to a local optimum. Note the empty-cluster reseed, the edge case interviewers probe.

```
"""Listing 10 -- k-means from scratch, NumPy only."""
import numpy as np

def kmeans(X, k, n_iter=100, seed=0):
    rng = np.random.default_rng(seed)
    # initialize centroids as k random distinct data points
    centroids = X[rng.choice(len(X), size=k, replace=False)]
    for _ in range(n_iter):
        # ASSIGN: nearest centroid for every point. (n,1,d)-(k,d) -> (n,k,d)
        d2 = ((X[:, None, :] - centroids[None, :, :]) ** 2).sum(axis=2) # (n, k)
        labels = d2.argmin(axis=1) # (n,)
        # UPDATE: each centroid = mean of its assigned points
        new_centroids = centroids.copy()
        for j in range(k):
            members = X[labels == j]
            if len(members):
                new_centroids[j] = members.mean(axis=0)
            else:
                # empty cluster: reseed at a random point
                new_centroids[j] = X[rng.integers(len(X))]
        if np.allclose(new_centroids, centroids): # assignments stable -> converged
            break
        centroids = new_centroids
    inertia = ((X - centroids[labels]) ** 2).sum() # within-cluster sum of squares
    return centroids, labels, inertia

rng = np.random.default_rng(7)
blobs = np.vstack([rng.normal(m, 0.5, (60, 2)) for m in [(0, 0), (4, 0), (2, 4)]])
centroids, labels, inertia = kmeans(blobs, k=3, seed=1)
print("centroids:\n", np.round(centroids[np.argsort(centroids[:, 0])], 2))
print("cluster sizes:", np.bincount(labels), " inertia:", round(inertia, 1))
```

Output:

```
centroids:
[[-0.09 -0.05]
 [ 1.97  4.02]
 [ 3.95 -0.1 ]]
cluster sizes: [60 60 60] inertia: 74.5
```


Listing 11 — Fisher-Yates shuffle and reservoir sampling

The `[i, n)` swap range is exactly what makes Fisher-Yates uniform — `[0, n)` is the famous biased variant. The empirical check confirms reservoir sampling's guarantee: every item lands in a $k=2$ -of-5 sample at $\sim 2/5 = 40\%$.

```

"""Listing 11 -- Fisher-Yates shuffle and reservoir sampling."""
import random
from collections import Counter

def fisher_yates(a, rng=random):
    """Uniform in-place shuffle: swap i with a random j in [i, n)."""
    for i in range(len(a) - 1):
        j = rng.randrange(i, len(a))  # NOT range(0, n) -- that version is biased
        a[i], a[j] = a[j], a[i]
    return a

def reservoir_sample(stream, k, rng=random):
    """Uniform k-sample from a stream of unknown length, O(k) memory."""
    reservoir = []
    for i, x in enumerate(stream, start=1):
        if i <= k:
            reservoir.append(x)  # keep the first k
        elif rng.random() < k / i:  # accept item i with prob k/i
            reservoir[rng.randrange(k)] = x  # evict a uniformly random resident
    return reservoir

random.seed(0)
print("shuffle:", fisher_yates(list(range(10))))
print("reservoir (k=3) from range(1000):", reservoir_sample(iter(range(1000)), 3))

# empirical uniformity check: each of 5 items should land in a k=2 sample ~40%
counts = Counter()
for _ in range(20000):
    counts.update(reservoir_sample(iter(range(5)), 2))
print("inclusion frequencies:", {i: round(c / 20000, 3) for i, c in
    sorted(counts.items())})

```

Output:

```

shuffle: [6, 7, 2, 5, 8, 4, 9, 3, 0, 1]
reservoir (k=3) from range(1000): [249, 584, 704]
inclusion frequencies: {0: 0.399, 1: 0.4, 2: 0.397, 3: 0.397, 4: 0.407}

```

Listing 12 — Weighted sampling and stratified splitting

Weighted draws = cumulative sums + `bisect` — the same mechanism as temperature sampling from a softmax. Stratified splitting is the hash-map grouping pattern applied to leakage-free evaluation: the 80/20 class ratio survives the split exactly.

```

"""Listing 12 -- Weighted sampling via CDF + binary search; stratified split."""
import bisect, random
from collections import Counter, defaultdict
from itertools import accumulate

def weighted_sampler(weights, rng=random):
    """O(n) setup, O(log n) per draw."""
    cdf = list(accumulate(weights))  # cumulative sums

```

```

total = cdf[-1]
def draw():
    u = rng.random() * total          # uniform in [0, total)
    return bisect.bisect_right(cdf, u) # first index with cdf > u
return draw

def stratified_split(labels, test_frac, rng=random):
    """Per-class index split -- preserves class proportions."""
    by_class = defaultdict(list)      # hash-map grouping pattern
    for i, y in enumerate(labels):
        by_class[y].append(i)
    train, test = [], []
    for y, idxs in by_class.items():
        rng.shuffle(idxs)
        cut = int(len(idxs) * test_frac)
        test += idxs[:cut]
        train += idxs[cut:]
    return sorted(train), sorted(test)

random.seed(0)
draw = weighted_sampler([0.1, 0.2, 0.7]) # e.g. softmax probabilities
freq = Counter(draw() for _ in range(10000))
print("empirical draw frequencies:", {i: round(c / 10000, 3) for i, c in
sorted(freq.items())})

labels = [0]*80 + [1]*20                # 80/20 imbalance
train, test = stratified_split(labels, test_frac=0.25)
print("test class balance:", Counter(labels[i] for i in test))
print("train class balance:", Counter(labels[i] for i in train))

```

Output:

```

empirical draw frequencies: {0: 0.103, 1: 0.192, 2: 0.705}
test class balance: Counter({0: 20, 1: 5})
train class balance: Counter({0: 60, 1: 15})

```

Pitfalls, comparisons and practical tips

Structure selection at a glance. The interview meta-skill is hearing a requirement and naming the structure in one step:

You need...	Use	Cost
"Have I seen this?" / count things	set / dict (Counter)	O(1) avg per op
Running min/max of a stream, top-k	heap	O(log k) per op
Process in arrival order / level order	deque (queue)	O(1) per op
Most-recent-first, nesting, undo	list (stack)	O(1) per op
Membership in <i>sorted</i> data, thresholds	sorted array + bisect	O(log n) lookup
Fewest steps between states	BFS	O(V+E)
Dependency ordering / cycle check	topological sort	O(V+E)
k-th smallest / median without sorting	quickselect / np.partition	O(n) avg

Complexity traps that cost offers. `list.pop(0)` and `list.insert(0, x)` are $O(n)$ — use `deque`. `x in my_list` is $O(n)$ inside a loop — that is an accidental $O(n^2)$; convert to a set once. String `+=` in a loop is $O(n^2)$ — collect parts, `''.join`. Slicing copies: `a[1:]` in a recursion turns $O(n)$ algorithms into $O(n^2)$ and is the classic hidden cost in tidy-looking recursive solutions — pass indices instead. `sorted()` inside a loop over n iterations is $O(n^2 \log n)$ — sort once outside, or maintain a heap.

Recursion traps. Python's default recursion limit ($\sim 1,000$) converts deep recursions into crashes: DFS on a long path graph, traversal of a degenerate tree, divide-and-conquer on adversarial input. The fixes, in preference order: rewrite with an explicit stack (Listing 5's DFS), convert the DP to bottom-up tabulation, or — last resort — raise the limit. Also remember Chapter 2's mutable-default trap: `def dfs(node, visited=set())` shares one set across *calls*, a bug that surfaces exactly in graph code.

Off-by-one discipline for binary search. Pick the half-open `[lo, hi)` convention and never mix it with inclusive bounds mid-function. Symptoms of mixing: infinite loops (`lo` never advances when `hi = mid` meets `mid = lo`), missing the first/last element, and index-out-of-range on empty inputs. `bisect_left` returns the *insertion point* — it may equal `len(a)`; check before indexing.

BFS vs DFS selection errors. Using DFS for "minimum number of steps" returns *a* path, not the shortest — a wrong-answer bug, not a crash, which makes it worse. Forgetting the visited set on a cyclic graph loops forever. Marking nodes visited when *popped* rather than when *pushed* lets BFS enqueue duplicates — correct answers, exponential queue.

Heap subtleties. `heapq` is min-only: negate for max behavior, and remember to negate again on the way out. Heap order is only guaranteed at the root — iterating a heap list does *not* yield sorted order; pop repeatedly or use `nsmallest`. Pushing unhashable/incomparable payloads without a tiebreaker key crashes on ties.

From-scratch ML implementation traps. In vectorized distance computation, the expansion $\|a-b\|^2 = \|a\|^2 + \|b\|^2 - 2a \cdot b$ can go slightly negative from floating-point error — clip at 0 before any `sqrt`. In k-means, forgetting the empty-cluster case crashes on `mean` of an empty slice; forgetting to seed the RNG makes results unreproducible, which interviewers notice. In KNN, taking `argsort(...)[k]` instead of `argpartition` is correct but signals you don't know selection beats sorting. In reservoir sampling, `k/i` with integer division (a Python 2 habit, or `//`) silently samples nothing past item `k`.

What interviewers actually score. Restate the problem and confirm edge cases (empty input, $k > n$, ties, cycles) *before* coding. Name the brute force and its complexity, then improve it — jumping straight to the clever solution without the baseline reads as memorization. State time and space complexity unprompted at the end. And for ML candidates specifically: connecting the pattern to its ML instance ("this is a top-k heap — same thing beam search does") is the highest-signal sentence you can say in this round.

Interview questions and answers

Arrays, strings, hash maps

Q1. You need to find whether any two numbers in an unsorted array sum to a target. Give three approaches and their trade-offs.

Brute force: check all pairs, $O(n^2)$ time, $O(1)$ space. Sort + two pointers: $O(n \log n)$ time, $O(1)$ extra space (beyond the sort), but destroys original indices. Hash map (one pass, ask for `target - x`): $O(n)$ time, $O(n)$ space, preserves indices. The hash map is the default answer; mention the two-pointer variant when the array is already sorted or memory is tight. *Interviewers listen for: the space-time trade-off stated explicitly, not just the fastest answer.*

Q2. Why is building a string with `+=` in a loop $O(n^2)$, and what is the fix?

Strings are immutable — each `+=` allocates a new string and copies both operands, so k appends of constant-size pieces copy $1+2+\dots+k$ characters: $O(n^2)$ total. Fix: append pieces to a list and `''.join(parts)` once — one final allocation, $O(n)$. (CPython has a narrow in-place optimization for unshared strings, but you must not rely on it and interviewers know it.)

Q3. Design an $O(n)$ algorithm for "longest substring without repeating characters."

Sliding window with a hash map of char $\rightarrow$ last index. Expand `right`; if `s[right]` was seen at position $\geq$ `left`, jump `left` to one past that position; track the max window length. Each pointer only moves forward, so $O(n)$ time, $O(\min(n, \text{alphabet}))$ space. The invariant to state: the window `[left, right]` never contains a repeat.

Q4. What makes a good dict key, and why can't you use a NumPy array or a list?

Keys must be hashable: define `__hash__` and `__eq__`, and the hash must never change while the key is in the table. Lists and arrays are mutable, so their hash would go stale and lookups would miss — both are unhashable by design. Convert to `tuple(arr)` or `arr.tobytes()` (include shape/dtype if they vary). This bites in practice when memoizing feature computations keyed by input arrays.

Q5. How would you find the top-10 most frequent tokens in a 100 GB text corpus that doesn't fit in memory?

Frequency counting doesn't need the corpus in memory — it needs the *vocabulary* in memory: stream the file, update a Counter (usually fits; vocabularies are far smaller than corpora), then `most_common(10)` (a size-10 heap over vocab entries, $O(V \log 10)$). If even the vocabulary overflows: hash-partition tokens across spill files so all occurrences of a token land in the same file, count each file separately, merge the per-file top-k candidates. *Interviewers listen for: distinguishing corpus size from vocab size, and the hash-partition trick.*

Stacks, queues, heaps**Q6. Check whether a string of brackets ()[]{} is balanced. What structure and why?**

A stack — nesting is LIFO. Push openers; on a closer, the stack top must be the matching opener (pop it), else fail. Valid iff the stack is empty at the end. Edge cases to name: closer on empty stack, leftover openers. $O(n)$ time, $O(n)$ worst-case space. Generalizes to expression parsing and to validating nested JSON/HTML.

Q7. Why does "k largest elements of a stream" use a min-heap rather than a max-heap?

You need fast access to the *smallest of the current top-k* — the candidate to evict when something better arrives. A min-heap of size k keeps that at the root: compare each new item to the root, `heappushpop` if larger, $O(\log k)$ per item and $O(k)$ memory. A max-heap of everything gives you the global max fast but eviction of the k -th best is what the problem needs. $O(n \log k)$ total vs $O(n \log n)$ for sorting.

Q8. How do you find the median of a data stream in $O(\log n)$ per element?

Two heaps: a max-heap of the lower half, a min-heap of the upper half, sizes kept within 1 of each other. Each insert goes into one heap and possibly rebalances one element across — $O(\log n)$; the median is a root (or the mean of both roots). This is the streaming form of quickselect's job, and the same two-heap idea underlies sliding-window medians for robust rolling statistics.

Q9. Why is deque the right structure for a BFS frontier and list the wrong one?

BFS pops from the front and pushes to the back. `deque.popleft()` is $O(1)$; `list.pop(0)` shifts every remaining element, $O(n)$, turning BFS's $O(V+E)$ into $O(V^2+E)$. A deque is a doubly-linked block structure with $O(1)$ at both ends. Same reasoning applies to any producer-consumer buffer, e.g. batches queued between `DataLoader` workers and a training loop.

Q10. Where exactly does a heap appear in beam search?

At each decoding step you have (beam $\times$ vocab) candidate extensions and need the k best by cumulative log-probability — a top- k selection. A size- k min-heap over candidate scores does it in $O(bV \log k)$; in practice frameworks use `topk` (partition-based selection) on the score tensor, which is the same operation vectorized. The heap framing also explains beam search's memory: $O(k)$ partial sequences, never the full search tree.

Trees, graphs

Q11. Why does inorder traversal of a BST yield sorted order, and what does that give you for validation?

The BST invariant puts everything smaller in the left subtree and everything larger in the right; inorder visits left subtree, node, right subtree, so by induction the output is sorted. Validation: run inorder and check the sequence is strictly increasing — $O(n)$, and it catches the deep-violation case (a node that satisfies its parent but violates a distant ancestor) that per-node child checks miss. The bounds-passing version (Listing 4) does the same without materializing the sequence.

Q12. Predicting with a depth- d decision tree is $O(d)$. Why can training-time tree depth blow up, and what limits it?

Prediction is one root-to-leaf descent: one threshold comparison per level. But an unconstrained tree keeps splitting until leaves are pure — on n distinct points it can grow to depth $O(n)$ (a chain) when splits are unbalanced, e.g. sorted or pathological features. Limits: `max_depth`, `min_samples_leaf`, `min_impurity_decrease` — the same hyperparameters that control overfitting (Chapter 6), because a memorizing tree and a degenerate tree are the same object.

Q13. When do you choose BFS over DFS and vice versa? Give the memory trade-off too.

BFS when the question is "fewest edges/steps" — it explores in distance layers, so the first arrival is optimal in an unweighted graph. DFS for exhaustive exploration: components, cycle detection, backtracking, topological orderings. Memory: BFS holds a frontier that can be $O(V)$ wide (bad on bushy graphs); DFS holds a path that can be $O(V)$ deep (bad on long chains, plus Python's recursion limit — use an explicit stack). Weighted shortest paths need Dijkstra: BFS with a priority queue.

Q14. Your feature pipeline is a set of "A must run before B" constraints. How do you order it, and how do you detect an impossible configuration?

Model as a DAG and topologically sort. Kahn's algorithm: repeatedly emit a zero-in-degree node and decrement its neighbors' in-degrees, using a queue; $O(V+E)$. If the algorithm stalls with nodes still un-emitted, those nodes form (or depend on) a cycle — the impossible configuration — and the leftover set localizes it for the error message. Same algorithm inside Airflow scheduling and autodiff's forward-pass ordering; backprop then runs the reverse order.

Q15. Count the connected components in a friendship graph with 10M nodes. What do you use and what breaks first?

Iterative DFS (or BFS) from every unvisited node, adjacency list, visited set: $O(V+E)$ time, $O(V)$ space. What breaks: recursive DFS hits Python's recursion limit on any long chain — must be an explicit stack; and at 10M nodes, per-object overhead of dict-of-lists may exhaust memory — move to array-based CSR adjacency or union-find (near- $O(1)$ amortized per edge), which also handles the streaming-edges version. *Interviewers listen for: the recursion-limit fix and one scaling escape hatch.*

Sorting, searching, DP**Q16. Why is quicksort $O(n \log n)$ average but $O(n^2)$ worst case, and how is the worst case avoided?**

Each partition costs $O(n)$; the total is $O(n \times \text{depth})$. Balanced partitions give depth $O(\log n)$; a pivot that is always the min/max (e.g. first element of sorted input) gives depth $O(n)$, hence $O(n^2)$. Mitigations: random pivot (adversarial inputs become astronomically unlikely), median-of-three, or introsort (quicksort that falls back to heapsort past a depth budget — what C++ `std::sort` does). Python sidesteps it entirely: Timsort is a mergesort hybrid with guaranteed $O(n \log n)$ and $O(n)$ on nearly-sorted data.

Q17. What is sort stability, and give a concrete ML-adjacent case where it matters.

A stable sort keeps equal keys in their original relative order. Case: event logs sorted by timestamp, then stable-sorted by `user_id` — each user's events remain time-ordered, which is required for building leakage-free sequential features or time-based splits; an unstable sort silently shuffles within-user order and corrupts them. Stability also composes multi-key sorts (secondary first, then primary). Python's `sorted` is stable; NumPy's default `np.sort` (quicksort) is not — pass `kind='stable'`.

Q18. You need the median of 1 billion floats. Full sort is too slow. Options?

Quickselect / `np.partition`: $O(n)$ average, in memory. Out of memory: (1) two-heap streaming works for running medians but still holds all data; (2) distribution/bucket approach — histogram pass to find the bucket containing the median, second pass within it; (3) approximate quantile sketches (t-digest, GK) with bounded memory and error guarantees — the production answer for percentile monitoring at scale.

Interviewers listen for: naming that exact selection is $O(n)$, then trading exactness for memory.

Q19. What property must hold to "binary search on the answer," and how would you find the largest classification threshold with recall ≥ 0.9 ?

Monotonicity of the feasibility predicate in the searched parameter. Recall is monotone non-increasing as the threshold rises (fewer positives predicted), so "recall ≥ 0.9 " flips from true to false exactly once — binary search the threshold in $[0, 1]$ (or over the sorted unique scores, which is exact) checking recall at each mid: $O(\log n)$ recall evaluations instead of scanning all thresholds. Precision, by contrast, is not monotone in the threshold — say so before anyone asks.

Q20. State the edit-distance recurrence and explain what each branch means.

$d(i, j) = d(i-1, j-1)$ if the current characters match (free), else $1 + \min(d(i-1, j), d(i, j-1), d(i-1, j-1))$ — delete from source, insert into source, substitute. Base cases $d(i, 0)=i$, $d(0, j)=j$. $O(mn)$ time; space compresses to two rows since each cell needs only the previous row and the current row's left neighbor. Used in spell-checkers, fuzzy joins in entity resolution; the same table shape as DTW, Viterbi, and CTC alignment.

Q21. Why is naive recursive Fibonacci exponential, and what are the two standard fixes? What's the general lesson?

$\text{fib}(n)$ branches into $\text{fib}(n-1)$ and $\text{fib}(n-2)$, recomputing the same subproblems exponentially often — the call tree has $\sim \phi^n$ nodes. Fixes: memoize the recursion (`@lru_cache` — top-down DP) or fill a table from the base cases (bottom-up, and keep only the last two values: $O(1)$ space). Lesson: when a recursion has overlapping subproblems and optimal substructure, caching collapses exponential to polynomial — the definition of DP.

Q22. Your recursive solution is correct but crashes with RecursionError on large inputs. Enumerate your options.

In order: (1) convert to iteration — explicit stack for traversals, bottom-up table for DP (removes the limit and is usually faster); (2) restructure to reduce depth (e.g. recurse into the smaller side only, as in quickselect); (3) `sys.setrecursionlimit` — a last resort that trades the clean exception for a possible interpreter crash and doesn't fix the $O(\text{depth})$ frame memory. Python has no tail-call optimization, so tail-recursive style does not help — worth saying explicitly.

ML from scratch, sampling

Q23. Walk through vectorizing KNN's distance computation. Why not two nested loops?

Expand $\|q-x\|^2 = \|q\|^2 + \|x\|^2 - 2q \cdot x$: compute row-norms of both matrices, one matrix multiply $Q @ X.T$, and broadcast the norms into a (queries $\times$ train) distance matrix — all C-loop work, $\sim 100\times$ faster than Python loops (Chapter 2's boxing/dispatch argument). Then `argsort` per row for the k smallest: $O(n)$ selection, not an $O(n \log n)$ sort. Caveats worth volunteering: float error can make entries slightly negative (clip at 0 before `sqrt`), and the $(q \times n)$ matrix itself can exceed memory — chunk the queries.

Q24. Why is k-means guaranteed to converge, and to what?

Both steps monotonically decrease (or leave unchanged) the within-cluster sum of squares: reassigning a point to a nearer centroid lowers its contribution, and the mean is the unique minimizer of summed squared distance to a set of points. The objective is bounded below and there are finitely many assignments, so the loop must reach a fixed point — but only a *local* optimum depending on initialization. Hence k-means++ seeding and `n_init` restarts (Chapter 8). Complexity per iteration: $O(nkd)$.

Q25. Prove reservoir sampling is uniform for $k = 1$.

Keep the first item; replace the current holder with item i with probability $1/i$. Item i survives to the end iff it is accepted ($1/i$) and never displaced by any later j (each j fails to displace with probability $1 - 1/j = (j-1)/j$). $P(\text{item } i \text{ final}) = (1/i) \times \prod_{j=i+1}^n (j-1)/j$ — the product telescopes to i/n — giving $1/n$ for every i . The $k>1$ proof is the same induction with k/i acceptance and uniform eviction. *Interviewers listen for: the telescoping product, stated cleanly.*

Q26. Implement weighted sampling given softmax probabilities. What are the complexity options?

Cumulative sums + binary search (Listing 12): $O(n)$ setup, $O(\log n)$ per draw — right for many draws from a fixed distribution, and exactly what temperature sampling over a vocabulary does. Alternatives: linear scan of the CDF, $O(n)$ per draw (fine for one draw); the alias method, $O(n)$ setup and $O(1)$ per draw (the answer to "can you beat $\log n$?"); for one-shot vectorized use, `np.random.choice(n, p=probs)`. If weights update between draws (boosting, prioritized replay), a Fenwick/segment tree gives $O(\log n)$ updates and draws.

Q27. Why is "shuffle by sorting with a random comparator" wrong, and what is the correct algorithm?

A random comparator is inconsistent (violates transitivity), so the sort's behavior is undefined and the resulting permutation distribution is biased — famously demonstrated in a browser-ballot shuffle. Sorting by a random key per element is unbiased but $O(n \log n)$ and can collide. Correct: Fisher-Yates — swap position i with uniform j in $[i, n)$, $O(n)$, exactly uniform by induction. The `[0, n)` variant is the subtle trap: it produces n^n equally likely execution paths, which cannot map uniformly onto $n!$ permutations since $n!$ does not divide n^n .

Q28. Your 80/20 imbalanced dataset needs a train/test split. What goes wrong with a plain random split and what's the fix?

With a small test set, the minority class's test count is a random variable — it can land far from 20% (or at zero for rare classes), making test metrics noisy or undefined (recall with no positives). Fix: stratified split — group indices by label (hash-map grouping), shuffle within groups (Fisher-Yates), take the fraction from each: class proportions hold exactly (Listing 12). Same idea extends to stratified k-fold CV; for time series, stratification gives way to temporal splits — leakage beats balance (Chapter 4).

Q29. Estimate: KNN prediction cost for 1M training points, 512-dim embeddings, 1,000 queries, k=10. Then make it fast.

Brute force: 10^9 point-pairs $\times$ 512 mult-adds $\approx 5 \times 10^{11}$ FLOPs — seconds as one big matrix multiply on good BLAS/GPU, but the $1000 \times 1\text{M}$ distance matrix is ~ 4 GB in float32, so chunk queries. Per query it's $O(nd)$, which kills low-latency serving.

Faster: KD/ball trees fail at $d=512$ (curse of dimensionality — no pruning); the real answers are approximate nearest neighbor — inverted-file indexes, product quantization, HNSW graphs (Chapter 24) — trading exact recall for orders-of-magnitude speedups. *Interviewers listen for: memory of the distance matrix, and knowing when trees stop working.*

Q30. Design a data structure supporting insert, delete, and get-random in $O(1)$, and say where the pattern shows up in ML.

Array + hash map: the array holds elements (random pick = uniform index, $O(1)$); the map holds value $\rightarrow$ array index. Delete: swap the target with the last element, update the moved element's map entry, pop — $O(1)$ because only the last slot ever vacates. The pattern (dense array for sampling + index map for lookup) is how replay buffers and negative-sampling pools support uniform draws with $O(1)$ membership updates.